RÉPUBLIQUE DU CAMEROUN

Paix - Travail - Patrie

UNIVERSITÉ DE YAOUNDÉ I

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE

DÉPARTEMENT DE GÉNIE INFORMATIQUE

REPUBLIC OF CAMEROON

Peace - Work - Fatherland

UNIVERSITY OF YAOUNDÉ I

NATIONAL ADVANCED SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

RAPPORT DE PROJET

Dans le cadre du stage au sein du laboratoire IoT

EyesHome

Dispositif de surveillance de paramètres d'une salle, connecté à une application Web

Stagiaires NOUMSI Elvira
NJOME David
TAGNE Lavoisier
TCHOUNOU Maeva
MEGNIJIO Lea

Encadreur Dr Anne Marie Chana

Année académique 2025 – 2026

Résumé

Les habitations et les petits locaux techniques sont exposés à des risques silencieux tels que les fuites de gaz, les variations anormales de température ou d'humidité, ou encore les intrusions non détectées, alors que les solutions de surveillance domestique existantes reposent le plus souvent sur une connexion Internet permanente et un abonnement à des services propriétaires, ce qui les rend coûteuses, dépendantes d'infrastructures tierces et inadaptées aux zones à couverture réseau instable ; c'est dans ce contexte que s'inscrit le projet EyesHome, mené dans le cadre d'un stage au laboratoire de l'Internet des objets de l'École Nationale Supérieure Polytechnique de Yaoundé, dont l'objectif a été de concevoir et de mettre en œuvre une chaîne complète de collecte, de transmission et de visualisation de données environnementales domestiques, en explorant successivement deux architectures de communication : une première approche fondée sur le protocole WiFi, permettant une validation rapide de la chaîne applicative, puis une seconde approche fondée sur la technologie LoRa/-LoRaWAN, mieux adaptée aux contraintes de portée et de consommation énergétique, intégrant un microcontrôleur ESP32 chargé de la lecture des capteurs, un module Wyres assurant la transmission radio, une passerelle LoRa, le serveur réseau ChirpStack et l'outil de traitement de flux Node-RED, le tout déployé entièrement en environnement local via Docker afin de garantir l'autonomie du système vis-à-vis de tout service en ligne, permettant la visualistion des données via une application web, mobile-first disponible uniquement dans cet environnement local. Les résultats obtenus avec le Wi-Fi démontrent la capacité du système à détecter en temps réel les changements significatifs de température, d'humidité, de concentration de gaz et de présence, à les transmettre de façon fiable jusqu'à une interface de visualisation, tout en mettant en évidence certaines limites liées à la portée radio effective et à la précision des capteurs à bas coût utilisés, ouvrant ainsi des perspectives d'amélioration vers l'ajout de capteurs plus performants, un système d'alerte plus élaboré et un déploiement à plus grande échelle. Les résultats obtenus avec le LoRaWAN démontrent quant à eux la capacité du système à conserver le même comportement, mais dans un environnement local, offrant ainsi plus de gestion et de contrôle sur les données collectées, tout en mettant en évidence certaines limites telles que la carte Wyres, qui rencontre des soucis de transmission des données, et l'accès aux fonctionnalités de visualisation, qui ne peut se faire qu'en local, ouvrant ainsi des perspectives vers l'utilisation d'une carte ESP32 avec module LoRa intégré.

Mots clés : Internet des objets (IoT), LoRa, LoRaWAN, ESP32, Wyres, Surveillance domestique, ChirpStack, Node-RED, Mobile-first

Abstract

Homes and small technical premises remain exposed to silent hazards such as gas leaks, abnormal temperature or humidity variations, and undetected intrusions, while most existing home monitoring solutions rely on a permanent Internet connection and proprietary cloud subscriptions, making them costly, dependent on third-party infrastructure, and poorly suited to areas with unreliable network coverage ; the EyesHome project, carried out as part of an internship at the IoT laboratory of the National Advanced School of Engineering of Yaoundé, addresses this issue by designing and implementing a complete chain for collecting, transmitting, and visualizing domestic environmental data, exploring two successive communication architectures : an initial WiFi-based approach that enabled rapid validation of the application chain, followed by a LoRa/LoRaWAN-based approach better suited to range and power-consumption constraints, combining an ESP32 microcontroller for sensor acquisition, a Wyres module for radio transmission, a LoRa gateway, the ChirpStack network server, and the Node-RED flow-processing tool, the whole solution being deployed entirely on-premises through Docker to ensure independence from any external online service, and enabling data visualization through a mobile-first web application available only within this local environment. The results obtained with WiFi demonstrate the system's ability to detect significant real-time changes in temperature, humidity, gas concentration, and presence, and to reliably transmit them to a visualization interface, while also revealing certain limitations related to effective radio range and the accuracy of the low-cost sensors used, opening perspectives for improvement through the addition of more advanced sensors, a more elaborate alerting system, and a larger-scale deployment. The results obtained with LoRaWAN demonstrate, in turn, the system's ability to maintain the same behaviour, but within a local environment, thereby offering greater management and control over the collected data, while also revealing certain limitations such as the Wyres board, which experiences data transmission issues, and access to visualization features, which can only be done locally, opening perspectives towards the use of an ESP32 board with an integrated LoRa module.

Keywords : Internet of Things (IoT), LoRa, LoRaWAN, ESP32, Wyres, Home monitoring, ChirpStack, Node-RED, Mobile-first

Abréviations et sigles

ADC Analog to Digital Converter (convertisseur analogique-numérique)

API Application Programming Interface (interface de programmation d'application)

ADR Adaptive Data Rate (adaptation dynamique du débit LoRaWAN)

AppEUI Application EUI (identifiant de l'application LoRaWAN)

AppKey Application Key (clé d'application utilisée lors de l'authentification OTAA)

AppSKey Application Session Key (clé de session applicative dérivée lors du join OTAA)

ASCII American Standard Code for Information Interchange

BLE Bluetooth Low Energy

CPU Central Processing Unit (processeur)

DevAddr Device Address (adresse dynamique attribuée au dispositif lors du join OTAA)

DevEUI Device EUI (identifiant unique du dispositif LoRaWAN)

Docker Plateforme de conteneurisation utilisée pour déployer localement les services

DHT11 Capteur numérique de température et d'humidité

ESP32 Microcontrôleur WiFi/Bluetooth utilisé pour l'acquisition des capteurs

GPIO General Purpose Input/Output (broche d'entrée/sortie polyvalente)

HTTP HyperText Transfer Protocol

IoT Internet of Things (Internet des objets)

JSON JavaScript Object Notation

LoRa Long Range (technique de modulation radio longue portée)

LoRaWAN Long Range Wide Area Network

MQ-2 Capteur de gaz combustible utilisé dans le prototype

MQTT Message Queuing Telemetry Transport

MPD Modèle Physique de Données

NwkSKey Network Session Key (clé de session réseau dérivée lors du join OTAA)

OTAA Over-The-Air Activation (procédure d'activation LoRaWAN)

PIR Passive InfraRed (capteur infrarouge passif de mouvement)

RX Receive / réception

STM32 Famille de microcontrôleurs ARM utilisée par le module Wyres

TX Transmit / émission

UART Universal Asynchronous Receiver-Transmitter

UML Unified Modeling Language

WiFi Wireless Fidelity

Wyres Module radio LoRaWAN utilisé comme passerelle de transmission entre l'ESP32 et le réseau LoRaWAN

Glossaire

Big-endian Convention d'encodage dans laquelle l'octet de poids fort est transmis avant l'octet de poids faible.

Classe A Classe LoRaWAN la plus économe en énergie : le dispositif est en veille permanente sauf lorsqu'il émet un uplink, et ne peut recevoir de downlink que dans deux fenêtres de réception ouvertes immédiatement après chaque émission.

Classe B Classe intermédiaire : le dispositif synchronise son horloge sur des balises émises par la gateway (beacon lock), ce qui lui permet d'ouvrir des fenêtres de réception périodiques programmées indépendamment de ses émissions.

Classe C Classe la plus énergivore mais offrant la latence descendante la plus faible : le dispositif maintient sa fenêtre de réception ouverte en permanence, sauf pendant ses propres émissions.

Complément à deux Méthode utilisée pour coder la température sur deux octets afin de représenter nativement les valeurs négatives sans octet de signe séparé.

Conteneur Docker Instance isolée permettant d'exécuter un service logiciel localement avec ses dépendances.

Détection de changement Mécanisme côté ESP32 qui déclenche une transmission uniquement lorsqu'une mesure varie au-delà d'un seuil défini ou lorsqu'un état change.

Device-profile Fichier de configuration décrivant les caractéristiques d'un dispositif LoRaWAN : classe d'opération (A, B ou C), fonction de décodage du payload, et paramètres réseau associés.

DevEUI Identifiant unique du module Wyres au sein du réseau LoRaWAN, enregistré lors du provisioning et associé à chaque trame reçue par ChirpStack.

Encodage hexadécimal Représentation des octets de mesure sous forme de caractères hexadécimaux avant l'envoi sur la liaison UART.

Gateway LoRa Passerelle qui reçoit les trames radio LoRaWAN et les relaie vers le serveur réseau.

JSON Format textuel structuré utilisé pour échanger des données entre l'ESP32, le serveur et l'application web lors de la phase WiFi.

Latence Délai entre la mesure d'une grandeur physique et son affichage dans l'interface de visualisation.

Mobile-first Approche de conception d'interface pensée d'abord pour les écrans mobiles.

Network server Composant de l'infrastructure LoRaWAN responsable de la gestion du réseau, de la déduplication des uplinks et de l'application du protocole MAC.

Node-RED flow Chaîne de nœuds visuels utilisée pour traiter, transformer et acheminer les données.

OTAA Procédure d'activation par voie radio au cours de laquelle le dispositif et le réseau échangent une requête et une acceptation de join, permettant de dériver dynamiquement les clés de session à partir de l'AppKey.

Payload Charge utile transmise en LoRaWAN, ici composée de sept octets contenant la température, l'humidité, le gaz et l'état de présence.

Seuil de changement Écart minimal de température, d'humidité ou de gaz, ou changement d'état de présence, au-delà duquel l'ESP32 déclenche l'envoi d'une nouvelle mesure.

Service-profile Profil définissant les paramètres de service associés à une gateway ou à une organisation : débits autorisés, politique de roaming, ajout des métadonnées gateway, etc.

Trame hexadécimale Chaîne de caractères ASCII représentant les octets de mesure encodés en base 16, échangée sur la liaison UART entre l'ESP32 et le module Wyres.

Uplink Dans ce projet, message envoyé du module Wyres vers la passerelle puis vers ChirpStack, contenant les données de capteurs.

Table des matières

Résumé **2**

Abstract **3**

Abréviations et sigles **4**

Glossaire **5**

Introduction **11**

1 Généralités et état de l'art **13**

1.1 Généralités . . . 13

1.1.1 Internet des objets et surveillance environnementale . . . 13

1.1.2 Le microcontrôleur ESP32 . . . 13

1.1.3 La technologie LoRa et le protocole LoRaWAN . . . 13

1.1.4 ChirpStack et Node-RED . . . 13

1.2 État de l'art . . . 14

1.2.1 Solutions existantes . . . 14

1.2.2 Synthèse comparative . . . 14

2 Méthodologie, Analyse et conception **15**

2.1 Méthodologie adoptée . . . 15

2.2 Analyse des besoins . . . 16

2.2.1 Besoins fonctionnels . . . 16

2.2.2 Besoins non fonctionnels . . . 17

2.3 Conception . . . 17

2.3.1 Diagramme de cas d'utilisation . . . 17

2.3.2 Diagramme de séquence . . . 18

2.3.3 Diagramme de déploiement . . . 20

3 Implémentation et résultats **21**

3.1 Implémentation . . . 21

3.1.1 Outils et technologies utilisés . . . 21

3.1.2 Matrice d'intégration matériel / logiciel . . . 22

3.1.3 Brochage du microcontrôleur ESP32 . . . 23

3.1.4 Format de la trame transmise au module Wyres . . . 24

3.1.5 Extrait de code source . . . 24

3.1.6 Association Passerelle-Chirpstack . . . 27

3.1.7 Configuration de ChirpStack et Node-RED . . . 27

3.1.8 Visualisation des données en BD . . . 30
3.2 Résultats obtenus . . . 31
3.2.1 Scénarios de test . . . 31
3.2.2 Captures d'écran et résultats visuels . . . 32

4 Difficultés, limites et perspectives **38**
4.1 Difficultés rencontrées lors de la réalisation du prototype . . . 38
4.1.1 Partage du port UART entre la console de débogage et la liaison ESP32–Wyres 38
4.1.2 Rejet des trames LoRaWAN après redémarrage . . . 38
4.1.3 Insertions multiples en base via Node-RED . . . 39
4.1.4 Incohérences de structure du payload transmis par l'intégration HTTP . . . 39
4.2 Limites du dispositif . . . 39
4.2.1 Restriction du duty cycle LoRaWAN . . . 39
4.2.2 Précision et portée des capteurs . . . 39
4.2.3 Broches insuffisantes sur la carte Wyres Base . . . 40
4.2.4 Boucle de réactivation de l'alarme . . . 40
4.2.5 Coût des alertes SMS . . . 40
4.3 Perspectives d'évolution . . . 40
4.3.1 Intégration d'un ESP32 avec module LoRa intégré . . . 40
4.3.2 Remplacement du bouton poussoir par une télécommande . . . 41
4.3.3 Utilisation de capteurs adaptés et de meilleure précision . . . 41
4.3.4 Évolution vers un réseau de surveillance multi-dispositifs . . . 41

Conclusion **42**

A Code source complet **44**
A.1 Code Source Complet ESP32 . . . 44
A.2 Code Source Complet de Wyres Base . . . 49

Liste des tableaux

3.1 Outils, technologies et composants matériels utilisés dans l'architecture 22
3.2 Matrice d'intégration des composants et outils du système 23
3.3 Brochage de la carte ESP32 23
3.4 Scénarios de test 31

Table des figures

2.1 Architecture du prototype basé sur la communication WiFi. 15
2.2 Architecture du prototype basé sur LoRa/LoRaWAN en environnement local. 16
2.3 Diagramme de cas d'utilisation 17
2.4 Diagramme de séquence technique du cas d'utilisation : Visualiser les mesures en temps réel 19
2.5 Diagramme de déploiement 20

3.1 Matériel utilisé 21
3.2 Brochage de la carte ESP32 24
3.3 Réception des données du device dans l'interface ChirpStack. 28
3.4 Détail d'une transmission LoRaWAN décodée dans ChirpStack. 29
3.5 Flux d'insertion des données dans Node-RED. 30
3.6 Modèle physique de données 30
3.7 Contenu de la base de données PostgreSQL 32
3.8 Capture d'ecran sans qu'on est aujouter un dispositif 33
3.9 Resultat avec le dispositif ajouter-1 34
3.10 Resultat avec le dispositif ajouter-2 35
3.11 Historique 36
3.12 Notifications 37

Introduction

L'Internet des objets connaît depuis une dizaine d'années une expansion continue, portée par la miniaturisation des capteurs, la baisse du coût des microcontrôleurs et la multiplication des protocoles de communication sans fil adaptés aux contraintes énergétiques des objets connectés. Ce mouvement touche en particulier le secteur du bâtiment intelligent, où la surveillance environnementale en temps réel devient un enjeu de sécurité et de confort aussi bien pour les habitations individuelles que pour les petits locaux techniques ou industriels. Dans les pays où la couverture réseau mobile reste inégale et où les coûts d'abonnement à des plateformes cloud propriétaires constituent un frein réel à l'adoption, la capacité à déployer un système de supervision autonome, économe en énergie et indépendant d'un fournisseur externe prend une importance particulière.

C'est dans ce cadre que s'inscrit le projet EyesHome, développé durant un stage au sein du laboratoire de l'Internet des objets de l'École Nationale Supérieure Polytechnique de Yaoundé. Les foyers et petits locaux techniques restent aujourd'hui largement dépourvus de systèmes de surveillance environnementale accessibles, la plupart des solutions commerciales existantes imposant soit une connexion WiFi permanente incompatible avec les zones mal couvertes, soit un abonnement à un service cloud tiers qui pose des questions de coût récurrent, de confidentialité des données et de dépendance vis-à-vis d'une infrastructure externe. Le problème central que ce projet cherche à résoudre est donc l'absence, dans ce contexte, d'une solution de surveillance domestique à la fois autonome, peu coûteuse et fonctionnant sans dépendance à un service en ligne permanent.

La problématique qui en découle peut se formuler ainsi : comment concevoir un système de collecte et de transmission de données environnementales domestiques qui soit fiable sur une portée significative, économe en énergie, et entièrement opérable en environnement local, sans recours à une infrastructure cloud tierce ?

Pour répondre à cette problématique, le projet s'est fixé les objectifs suivants, formulés selon une logique SMART : concevoir, dans un délai de stage donné, un prototype matériel intégrant au moins quatre types de capteurs (température, humidité, gaz, présence) autour d'un microcontrôleur ESP32 ; établir une première chaîne de transmission fonctionnelle par WiFi vers un serveur en ligne, mesurable par la réception effective des données sur une application web ; migrer cette chaîne vers une architecture LoRa/LoRaWAN mesurable par la réception des mêmes données via une passerelle radio et le serveur réseau ChirpStack ; et finaliser un déploiement entièrement local, vérifiable par le fonctionnement de l'ensemble de la chaîne (base de données, Node-RED, application de visualisation) sans connexion à un service externe.

La méthodologie adoptée a suivi une démarche incrémentale en deux grandes phases. La première phase a consisté à établir un prototype de validation reposant sur une communication WiFi entre la carte ESP32 et un script hébergé sur un serveur en ligne, permettant de vérifier rapidement la cohérence de la chaîne de collecte, de transmission et d'affichage des données, sans les contraintes propres à la radio LoRa. La seconde phase a consisté à faire évoluer cette architecture vers une communication LoRa/LoRaWAN, impliquant un module émetteur Wyres relié à l'ESP32 par liaison UART, une passerelle LoRa réceptrice, le serveur réseau ChirpStack pour la gestion du réseau LoRaWAN et

le décodage des trames, et l'outil Node-RED pour le traitement et l'acheminement des données vers l'application finale ;

Le présent rapport est structuré en trois chapitres. Le premier chapitre présente les généralités nécessaires à la compréhension du projet ainsi qu'un état de l'art des solutions existantes de surveillance domestique connectée. Le deuxième chapitre détaille la méthodologie adoptée, l'analyse des besoins et la conception du système. Le troisième chapitre décrit l'implémentation réalisée, les outils utilisés et les résultats obtenus, avant qu'une conclusion générale ne revienne sur les objectifs atteints et les perspectives d'évolution du projet.

Chapitre 1

Généralités et état de l'art

1.1 Généralités

1.1.1 Internet des objets et surveillance environnementale

L'Internet des objets désigne l'ensemble des dispositifs physiques capables de collecter des données de leur environnement, de les traiter localement ou à distance, et de les transmettre à travers un réseau de communication. Dans le contexte d'une habitation, un système de surveillance environnementale connecté repose typiquement sur trois éléments : des capteurs chargés de mesurer des grandeurs physiques telles que la température, l'humidité ou la concentration d'un gaz, un microcontrôleur assurant l'acquisition et le pré-traitement de ces mesures, et un canal de communication permettant d'acheminer les données vers une interface de visualisation ou de stockage.

1.1.2 Le microcontrôleur ESP32

L'ESP32 est un microcontrôleur à double cœur intégrant nativement un module WiFi et Bluetooth [6], largement utilisé dans les projets d'électronique embarquée grâce à son faible coût, à la richesse de ses broches d'entrée/sortie et à la disponibilité d'un environnement de développement Arduino simplifiant l'écriture du code. Dans ce projet, l'ESP32 assure la lecture des capteurs analogiques et numériques ainsi que la communication, selon la phase du projet, soit directement par WiFi, soit par liaison série UART vers un module de transmission LoRa.

1.1.3 La technologie LoRa et le protocole LoRaWAN

LoRa est une technique de modulation radio à spectre étalé permettant des communications sur de longues distances (plusieurs kilomètres en zone dégagée) avec une consommation énergétique très réduite, au prix d'un débit de données faible. LoRaWAN est le protocole de couche réseau construit au-dessus de la modulation LoRa, définissant la manière dont les objets connectés (appelés nœuds) communiquent avec une ou plusieurs passerelles, lesquelles retransmettent les messages vers un serveur réseau chargé de la gestion des sessions, de la sécurité et du routage des données vers les applications finales [1].

1.1.4 ChirpStack et Node-RED

ChirpStack est une suite logicielle open source assurant les fonctions de serveur réseau LoRaWAN : gestion des passerelles, authentification et déchiffrement des trames des objets connectés, et exposition des données décodées via différents protocoles (HTTP, MQTT) [4]. Node-RED est un

outil de programmation par flux permettant de connecter facilement des sources de données, comme les messages émis par ChirpStack, à des traitements et des interfaces de visualisation, sans nécessiter le développement d'une application complète pour chaque nouveau besoin [5].

1.2 État de l'art

1.2.1 Solutions existantes

Plateformes domotiques commerciales

Les plateformes domotiques commerciales grand public proposent des capteurs pré-configurés (température, fumée, mouvement) associés à une application mobile et à un service cloud propriétaire. Leur principal avantage réside dans la simplicité d'installation et l'expérience utilisateur soignée. Leurs limites tiennent cependant à la dépendance systématique à une connexion Internet stable, à des frais d'abonnement récurrents pour l'accès aux fonctionnalités avancées, et à l'absence de maîtrise sur l'hébergement et la confidentialité des données collectées.

Projets IoT open source à base de WiFi

De nombreux projets communautaires proposent des solutions de surveillance basées sur des microcontrôleurs ESP32 ou ESP8266 communiquant en WiFi avec un serveur auto-hébergé ou un service cloud gratuit. Ces solutions offrent un bon niveau de personnalisation et un coût matériel réduit. Leur limite principale reste la portée du WiFi, généralement insuffisante pour couvrir un bâtiment complet ou une installation extérieure, ainsi qu'une consommation énergétique qui rend difficile un fonctionnement prolongé sur batterie.

1.2.2 Synthèse comparative

Le projet EyesHome se positionne à la croisée de ces deux catégories de solutions : il conserve la simplicité de mise en œuvre et le faible coût matériel des projets open source à base d'ESP32, tout en visant, grâce à la migration vers LoRa/LoRaWAN, une portée de communication nettement supérieure à celle du WiFi et une consommation énergétique compatible avec un fonctionnement prolongé. Contrairement aux plateformes commerciales, l'ensemble de la chaîne logicielle finale (serveur réseau ChirpStack, traitement Node-RED, base de données et application de visualisation) est déployé localement via Docker, garantissant l'indépendance du système vis-à-vis de tout service en ligne tiers et une maîtrise complète des données collectées.

Chapitre 2

Méthodologie, Analyse et conception

2.1 Méthodologie adoptée

Le projet a été mené selon une démarche incrémentale en deux grandes étapes, chacune constituant un jalon fonctionnel validé avant de passer à la suivante.

La **première étape** a consisté à établir un prototype de validation reposant sur une communication WiFi entre la carte ESP32 et un script hébergé sur un serveur en ligne. Cette étape a permis de vérifier rapidement, sans les contraintes de la radio LoRa, que la chaîne complète de lecture des capteurs, de mise en forme des données au format JSON, de transmission par requête HTTP et d'affichage sur une interface distante fonctionnait correctement de bout en bout.

Phase 1 — Prototype basé sur la communication WiFi

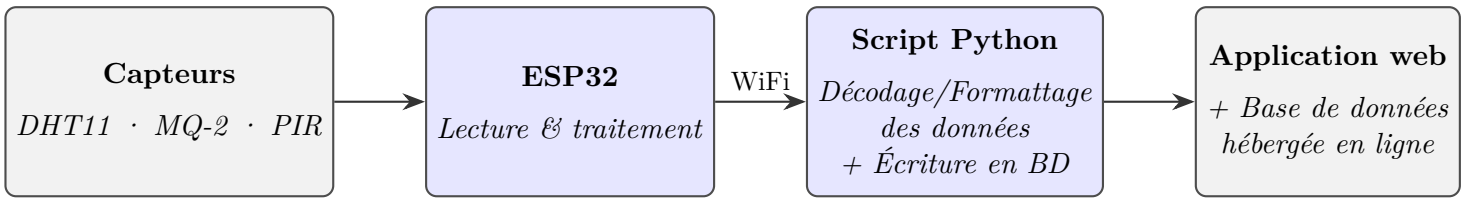


FIGURE 2.1 – Architecture du prototype basé sur la communication WiFi.

La **seconde étape** a consisté à faire évoluer cette architecture vers une communication LoRa/-LoRaWAN. L'ESP32 a été relié par liaison UART à un module Wyres jouant le rôle d'émetteur LoRa, lequel transmet les données jusqu'à une passerelle LoRa, puis au serveur réseau ChirpStack chargé du décodage des trames, et enfin à Node-RED pour le traitement et l'acheminement vers l'application de visualisation. Cette seconde étape s'est conclue par une **phase de finalisation**, au cours de laquelle l'ensemble des composants logiciels, y compris Node-RED, ont été déployés en environnement entièrement local via Docker.

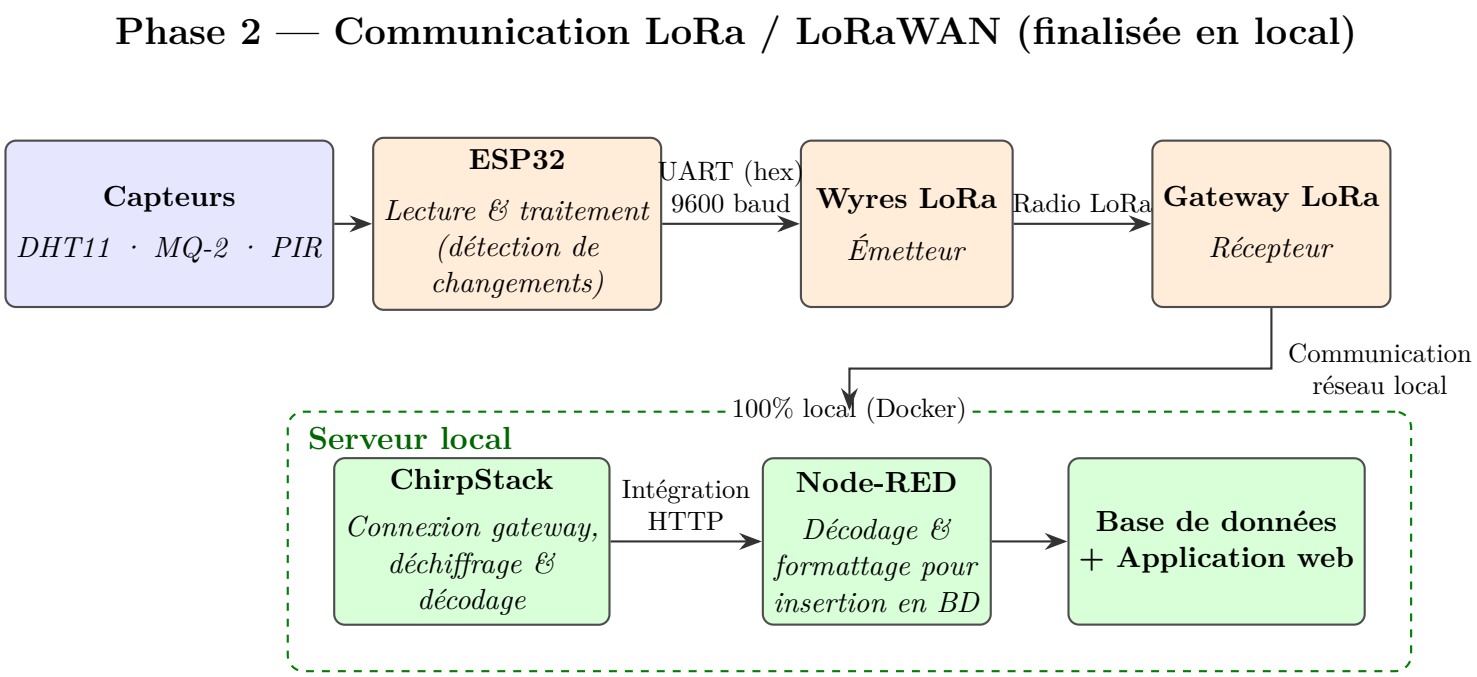


FIGURE 2.2 – Architecture du prototype basé sur LoRa/LoRaWAN en environnement local.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

- **Mesure continue :** Le système doit permettre de mesurer en continu les différents paramètres observés.
- **Visualisation des données :** Transmettre ces données vers une interface de visualisation accessible sur mobile.
- **Optimisation des transmissions :** Limiter les transmissions radio aux seuls changements significatifs afin de préserver la bande passante et l'énergie du système.
- **Détection de gaz :** Déterminer la concentration de certains gaz dans l'environnement direct.
- **Détection de présence :** Détecter une présence dans la zone surveillée.
- **Prise de température ambiante et de pourcentage d'humidité :** Déterminer la température ambiante et le pourcentage d'humidité de l'environnement direct.

2.2.2 Besoins non fonctionnels

- **Supervision quasi temps réel :** Présenter une latence de transmission compatible avec une supervision quasi temps réel.
- **Efficacité énergétique :** Consommation énergétique réduite côté nœud capteur.
- **Portée étendue :** Portée de communication significativement supérieure à celle d'une liaison WiFi domestique.
- **Robustesse :** Robustesse face aux erreurs de lecture ponctuelles des capteurs.
- **Isolation des données :** Données isolées et contrôlées par une implémentation en réseau local.

2.3 Conception

2.3.1 Diagramme de cas d'utilisation

L'utilisateur du système interagit avec EyesHome selon sept cas d'utilisation principaux, répartis en deux catégories : la consultation (visualiser les mesures en temps réel, consulter l'historique des alertes, télécharger le rapport journalier des données), la configuration (modifier les configurations d'alertes) et la gestion des dispositifs (ajouter, modifier et supprimer).

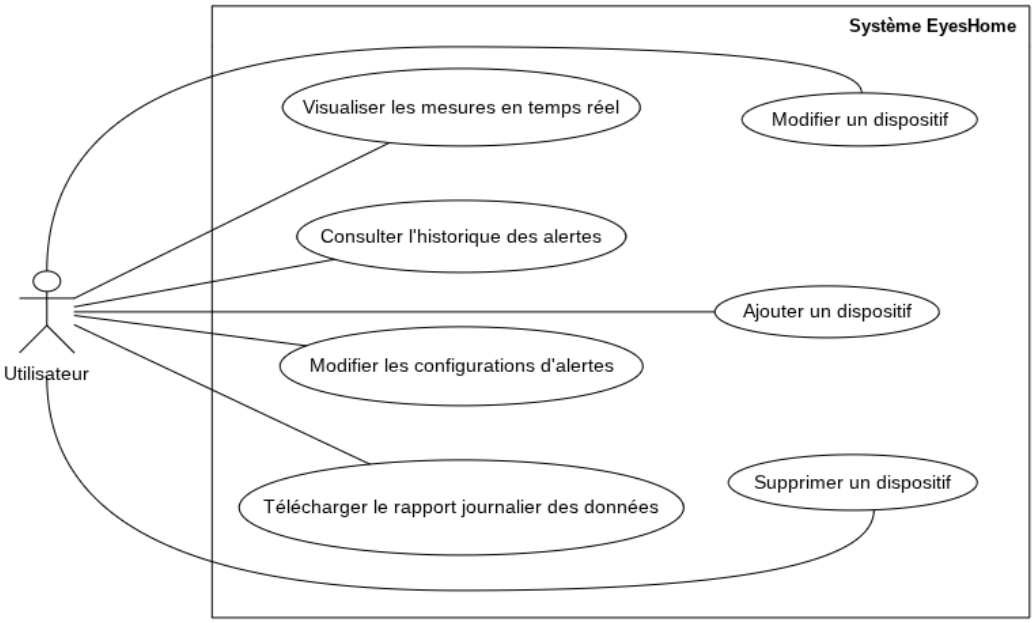


FIGURE 2.3 – Diagramme de cas d'utilisation

2.3.2 Diagramme de séquence

Le diagramme illustre le cas d'utilisation « Visualiser les mesures en temps réel ». L'utilisateur ouvre la page de visualisation, ce qui déclenche une requête vers l'application, une lecture de la table sensor_readings et un premier affichage. Ensuite, à chaque changement significatif détecté, l'ESP32 génère une trame transmise via UART au module Wyres LoRa, puis en LoRaWAN à la gateway. ChirpStack décode la trame et transmet le payload à Node-RED via HTTP, qui l'écrit en base après extraction des données utiles. L'application est alors notifiée et met à jour l'affichage.

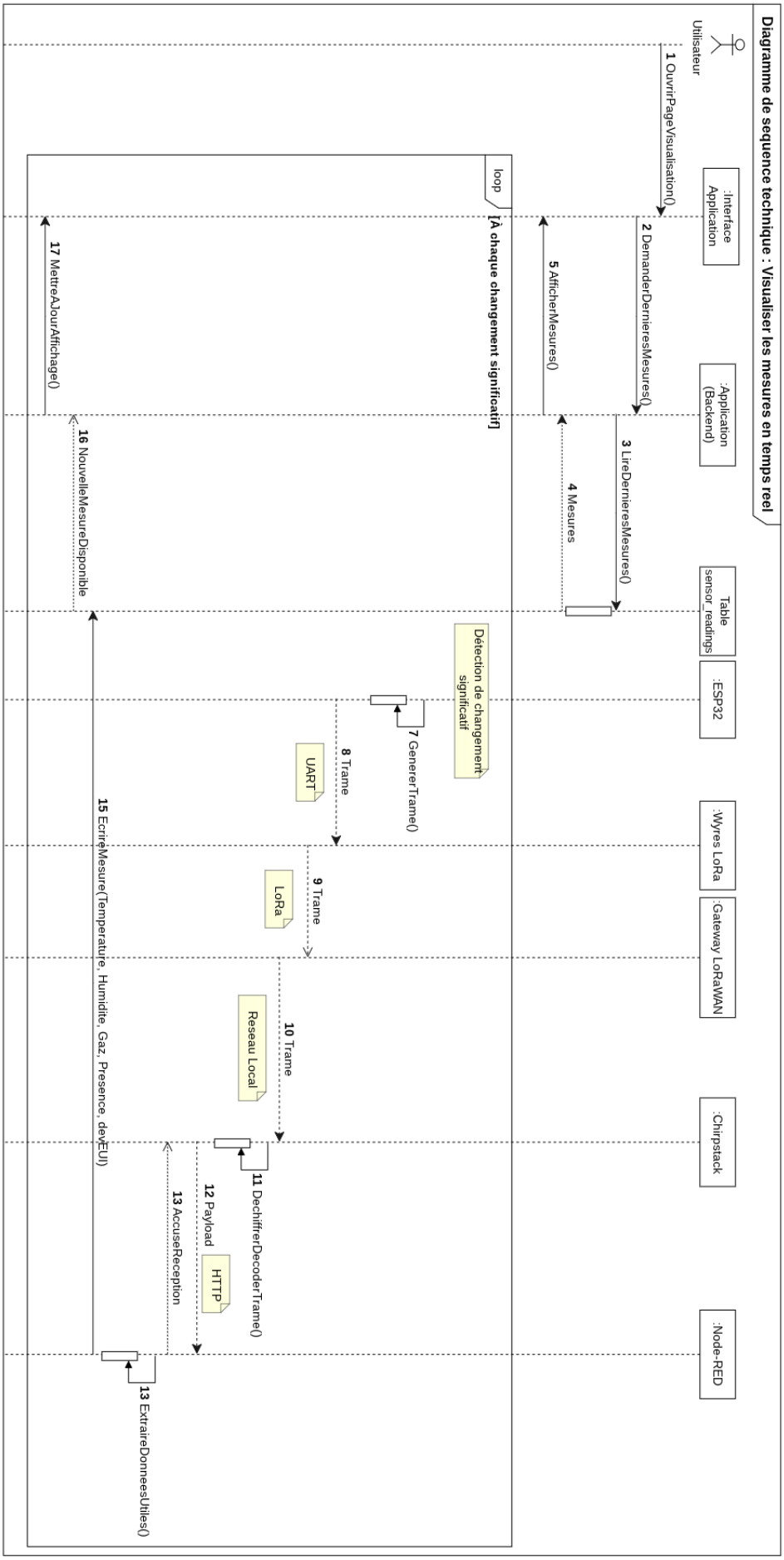


FIGURE 2.4 – Diagramme de séquence technique du cas d'utilisation : Visualiser les mesures en temps réel

2.3.3 Diagramme de déploiement

Le diagramme de déploiement de la figure 2.5 présente l'architecture physique et logicielle du système. Sur le plan matériel, un nœud capteur regroupe les capteurs DHT11, MQ-2 et PIR, connectés à un nœud ESP32 qui assure la lecture, le traitement et la détection de changements significatifs. Ce dernier est relié par UART (hexadécimal, 9600 baud) à un nœud Wyres LoRa, qui émet en radio LoRa vers un nœud Gateway LoRaWAN. La passerelle communique avec le serveur local via le réseau local. Ce serveur héberge, dans des conteneurs Docker, trois artefacts logiciels : ChirpStack (connexion à la passerelle, déchiffrage et décodage des trames), Node-RED (décodage et formattage pour l'insertion en base) et un ensemble Base de données + Application web. L'ensemble est déployé en 100% local, sans dépendance à une infrastructure cloud externe.

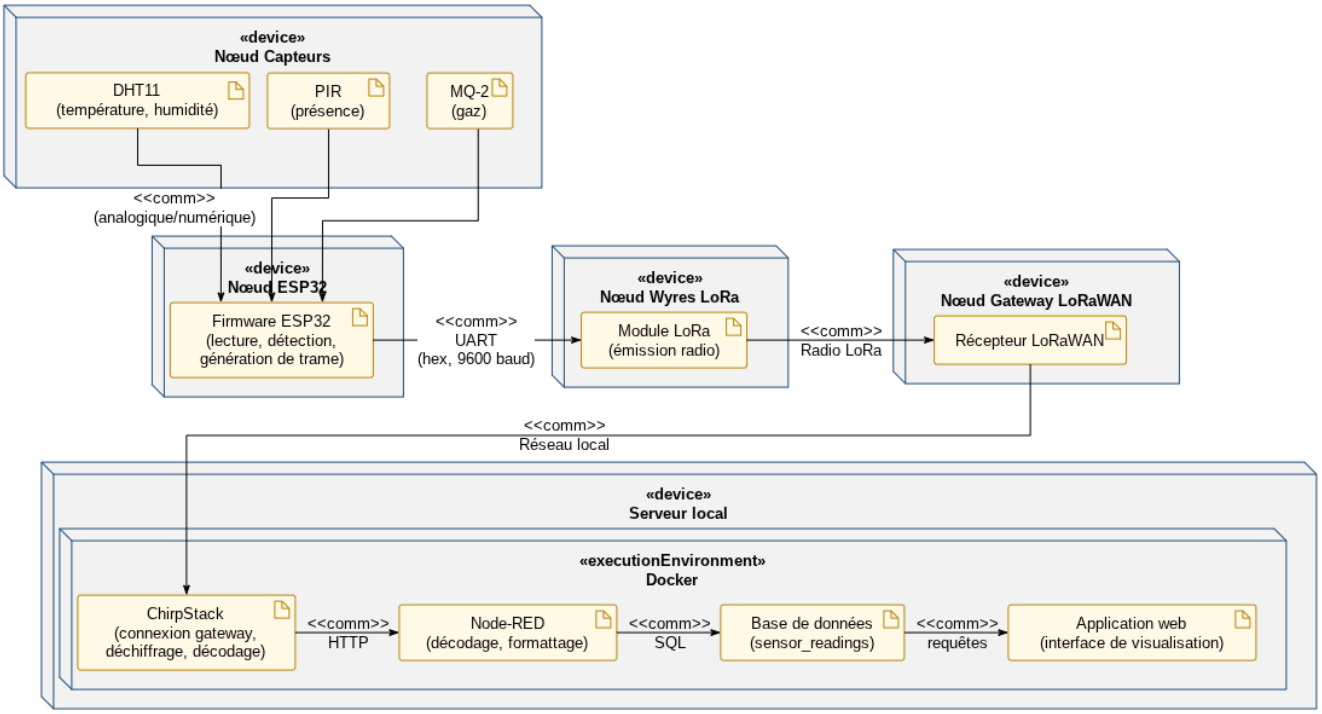


FIGURE 2.5 – Diagramme de déploiement

Chapitre 3

Implémentation et résultats

3.1 Implémentation

3.1.1 Outils et technologies utilisés

Le tableau 3.1 présente les principaux outils et technologies mobilisés au cours du projet, ainsi que leur rôle respectif dans la chaîne globale.

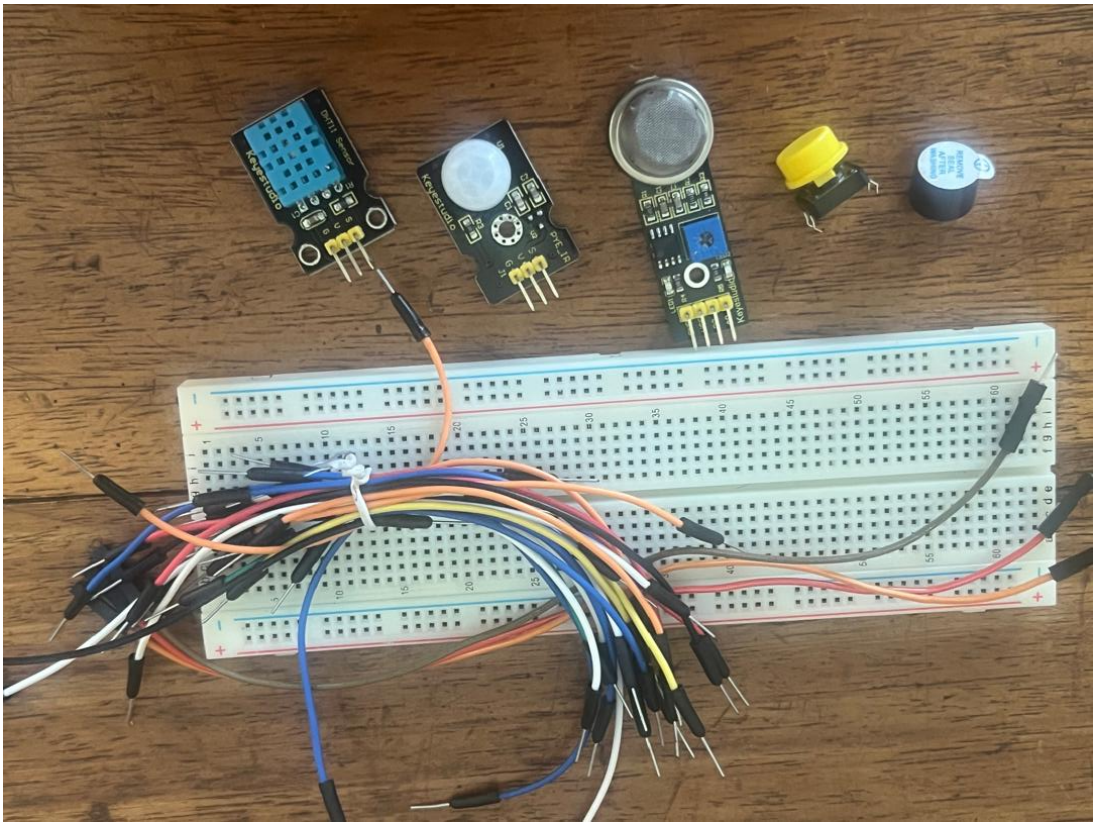


FIGURE 3.1 – Matériel utilisé

Outil / Technologie	Rôle dans le système
ESP32	Lecture des capteurs (température, humidité, gaz, présence), détection des changements significatifs et encodage des mesures avant transmission
Capteur DHT11	Mesure de la température et de l'humidité ambiantes
Capteur MQ-2	Détection de la concentration de gaz dans l'environnement surveillé
Capteur PIR	Détection de mouvement / présence humaine
Buzzer	Émission d'un signal sonore d'alerte locale en cas de dépassement de seuil critique
Bouton poussoir	Acquittement manuel de l'alerte pour arrêter le signal sonore du buzzer
Module Wyres (STM32)	Émission des données au format LoRaWAN vers la passerelle radio
Passerelle LoRaWAN	Réception des trames radio et retransmission vers le serveur réseau
ChirpStack	Serveur réseau LoRaWAN : authentification du dispositif, décodage du payload et exposition des données
Node-RED	Traitement des flux de données décodées et acheminement vers l'application de visualisation
Docker	Conteneurisation et déploiement local de l'ensemble des services logiciels
Arduino IDE	Environnement de développement et de programmation de l'ESP32
Next.js	Framework React utilisé pour le frontend et comme serveur d'API pour le backend
Nodemailer	Envoi d'emails via SMTP
jsPDF	Génération du rapport journalier PDF

TABLE 3.1 – Outils, technologies et composants matériels utilisés dans l'architecture

3.1.2 Matrice d'intégration matériel / logiciel

Le tableau 3.2 précise, pour chaque outil ou composant, s'il relève d'un élément matériel ou logiciel de l'architecture globale.

Outil / Technologie	Rôle principal	Matériel	Logiciel
ESP32	Lecture capteurs, détection et encodage des mesures	✓	×
Capteur DHT11	Mesure température / humidité	✓	×
Capteur MQ-2	Détection de gaz	✓	×
Capteur PIR	Détection de présence	✓	×
Buzzer	Alerte sonore locale	✓	×
Bouton poussoir	Acquittement / coupure buzzer	✓	×
Module Wyres	Émission radio LoRaWAN	✓	×
Passerelle LoRaWAN	Réception et relai des trames	✓	×
ChirpStack	Serveur réseau LoRaWAN	×	✓
Node-RED	Traitement des flux de données	×	✓
Docker	Déploiement local des services	×	✓
Arduino IDE	Programmation du firmware ESP32	×	✓

TABLE 3.2 – Matrice d'intégration des composants et outils du système

3.1.3 Brochage du microcontrôleur ESP32

Le tableau 3.3 récapitule l'affectation des broches utilisées sur la carte ESP32 pour la lecture des capteurs et la communication avec le module Wyres.

Broche (GPIO)	Fonction
GPIO 23	Capteur DHT11 (température / humidité)
GPIO 34	Capteur MQ-2 (gaz, entrée analogique)
GPIO 25	Capteur PIR (présence)
GPIO 32	Bouton poussoir (arrêt buzzer)
GPIO 33	Buzzer
GPIO 16	UART2 — Réception (RX) depuis le module Wyres
GPIO 17	UART2 — Émission (TX) vers le module Wyres

TABLE 3.3 – Brochage de la carte ESP32

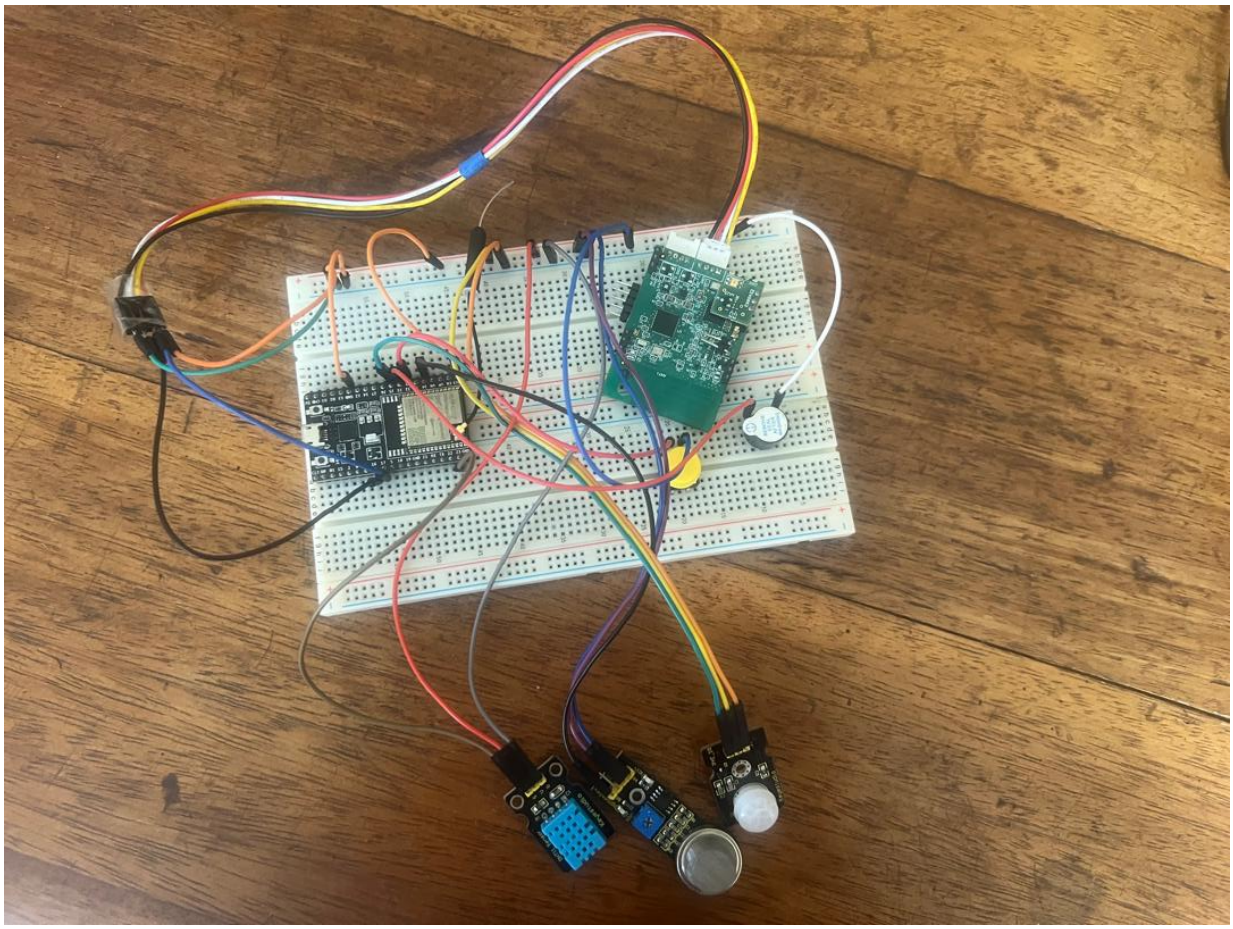


FIGURE 3.2 – Brochage de la carte ESP32

3.1.4 Format de la trame transmise au module Wyres

La trame transmise par liaison UART entre l'ESP32 et le module Wyres est une chaîne de quatorze caractères hexadécimaux représentant sept octets, structurée comme suit : les deux premiers octets encodent la température multipliée par dix en complément à deux, les deux octets suivants encodent l'humidité multipliée par dix, les deux octets suivants encodent le pourcentage de gaz multiplié par dix, et le dernier octet encode l'état de présence (0 ou 1), l'ensemble des valeurs multi-octets étant transmis en ordre big-endian. La multiplication par dix permet de conserver une décimale sans recourir à un format flottant, plus coûteux à transmettre et à parser sur un microcontrôleur ; le complément à deux permet, lui, de représenter des valeurs négatives, indispensables pour la température. Enfin, l'encodage sur deux octets plutôt qu'un seul est nécessaire pour couvrir les plages de mesure réelles : sur un seul octet non signé, une valeur multipliée par dix serait plafonnée à 25,5 (puisque $255/10 = 25,5$), ce qui est insuffisant pour la température, l'humidité comme pour le pourcentage de gaz.

3.1.5 Extrait de code source

Cette section présente les extraits de code les plus significatifs des deux microcontrôleurs impliqués dans la chaîne de transmission. Le code complet est disponible en annexe.

Code de l'ESP32 (Arduino)

Le code de l'ESP32 assure trois rôles : la détection de changement significatif, l'encodage hexadécimal des mesures et l'envoi de la trame au module Wyres via UART. Seules les portions essentielles à cette architecture sont reproduites ci-dessous.

Détection de changement : La boucle principale compare chaque mesure aux seuils définis et aux valeurs précédemment envoyées. Un envoi n'est déclenché que si au moins un paramètre a varié de manière significative.

Listing 3.1 – Détection de changement dans la boucle principale

```
#define SEUIL_TEMP     1.0    //  C    : envoi si   cart   >= 1 C
#define SEUIL_HUM      5.0    // %    : envoi si   cart   >= 5%
#define SEUIL_GAZ      10     // %    : envoi si   cart   >= 10%

bool changement = premierEnvoi;

if (fabs(temperature - temp_precedente) >= SEUIL_TEMP) changement = true;
if (fabs(humidite - hum_precedente) >= SEUIL_HUM) changement = true;
if (abs(pourcentageGaz - gaz_precedent) >= SEUIL_GAZ) changement = true;
if (presence != presence_precedente) changement = true;
```

Encodage hexadécimal : Chaque grandeur est multipliée par dix, convertie en entier signé ou non signé, puis placée sur deux octets (ou un seul pour la présence) en respectant l'ordre big-endian.

Listing 3.2 – Encodage des mesures en mots de 16 bits

```
int16_t temp_signed = (int16_t)constrain((int)round(temp * 10), -32768, 32767);
uint16_t hum_word = (uint16_t)constrain((int)round(hum * 10.0), 0, 65535);
uint16_t gaz_word = (uint16_t)constrain((int)round(gaz * 10.0), 0, 65535);
uint8_t presence_byte = motion ? 1 : 0;
```

Construction et envoi de la trame : Les quatre valeurs sont converties en caractères hexadécimaux puis concaténées en une trame de quatorze caractères, transmise sur le second port série.

Listing 3.3 – Assemblage et émission de la trame UART

```
word16ToHex(temp_word, hexTemp);
word16ToHex(hum_word, hexHum);
word16ToHex(gaz_word, hexGaz);
byteToHex(presence_byte, hexPresence);

String trame = String(hexTemp) + String(hexHum) + String(hexGaz) + String(
    hexPresence);
Serial2.println(trame);
```

Code de la carte Wyres (C)

Le code du module Wyres assure la réception de la trame, sa validation, son décodage en octets puis l'envoi du payload via LoRaWAN [3]. Seules les portions essentielles à cette architecture sont reproduites ci-dessous.

Réception et validation de la trame : Le thread de réception assemble la trame depuis le tampon circulaire, vérifie sa longueur minimale et s'assure que tous les caractères sont bien hexadécimaux avant de notifier le thread principal.

Listing 3.4 – Réception et validation de la trame hexadécimale

```
if (rx_buf_len < 14) continue;
bool valid = true;
for (uint16_t i = 0; i < rx_buf_len; i++) {
    char ch = rx_buf[i];
    if (!((ch >= '0' && ch <= '9') ||
          (ch >= 'A' && ch <= 'F') ||
          (ch >= 'a' && ch <= 'f'))) {
        valid = false;
        break;
    }
}
if (!valid) continue;
```

Décodage big-endian : Une fois la trame validée, chaque paire de caractères hexadécimaux est convertie en octet, puis les octets sont recombinés en valeurs 16 bits selon l'ordre big-endian.

Listing 3.5 – Décodage big-endian des sept octets

```
int16_t  temp = (int16_t)((payload_buf[0] << 8) | payload_buf[1]);
uint16_t hum  = (uint16_t)((payload_buf[2] << 8) | payload_buf[3]);
uint16_t gaz  = (uint16_t)((payload_buf[4] << 8) | payload_buf[5]);
uint8_t  pres =  payload_buf[6];
```

Envoi LoRaWAN : Le payload est transmis via la pile LoRaWAN, avec gestion des erreurs et nouvelle tentative en cas d'échec.

Listing 3.6 – Envoi du payload LoRaWAN avec gestion des erreurs

```
uint8_t res = semtech_loramac_send(&loramac, payload_buf, payload_len);

switch (res) {
    case SEMTECH_LORAMAC_TX_DONE:
        tx_ok++;
        return true;
    case SEMTECH_LORAMAC_DUTYCYCLE_RESTRICTED:
        ztimer_sleep(ZTIMER_SEC, 30);
        break;
    case SEMTECH_LORAMAC_NOT_JOINED:
```

```
            lorawan_join();
            break;
        default:
            ztimer_sleep(ZTIMER_MSEC, LORA_RETRY_DELAY_MS);
            break;
}
```

3.1.6 Association Passerelle-Chirpstack

C'est ici que commence véritablement la mise en place du réseau local utilisé pour la transmission des données jusqu'à l'interface de visualisation.

Mise en réseau local de la passerelle et du serveur

La gateway LoRaWAN et le serveur local sont mis en réseau local grâce à un modem. La gateway est configurée, à travers son interface web graphique, pour se connecter à ce réseau local en lui fournissant la clé de connexion. Par le même moyen, elle est configurée pour envoyer toutes les trames LoRa qu'elle capte vers le port du serveur réseau de ChirpStack, sur le même réseau local [2]. C'est principalement de cette configuration que découle le fait que les deux équipements soient en réseau.

Configuration initiale de ChirpStack

Une fois le conteneur Docker contenant toute la stack applicative lancé, la configuration de ChirpStack sur le serveur local commence par la création des éléments génériques suivants :

— une instance de **network server** ;

— un **service-profile** ;

— une **application**, qui regroupe une flotte de dispositifs poursuivant le même but.

Enregistrement de la gateway

La première configuration précise consiste à ajouter une gateway dans ChirpStack. C'est à cette étape que se finalise la communication entre la gateway et ChirpStack : il s'agit de renseigner ChirpStack avec les identifiants de la gateway afin qu'il reconnaisse et accepte tout message provenant de ces identifiants.

3.1.7 Configuration de ChirpStack et Node-RED

Création du device-profile

Le device-profile définit les caractéristiques qu'un dispositif aura. Dans notre cas précis, il faut y préciser :

— la **fonction de décodage** du payload reçu, correspondant à l'encodage défini au départ : les deux premiers octets pour la température, les deux suivants pour l'humidité, les deux suivants pour le gaz, et le dernier octet pour la présence ;

— la **classe** du dispositif, qui sera la classe C ;

— le **mode d'activation**, qui sera OTAA.

Ajout du dispositif

Le dispositif est ensuite ajouté proprement dans l'application créée au préalable, en renseignant le device-profile, le **DevEUI** et l'**AppKey**, entre autres.

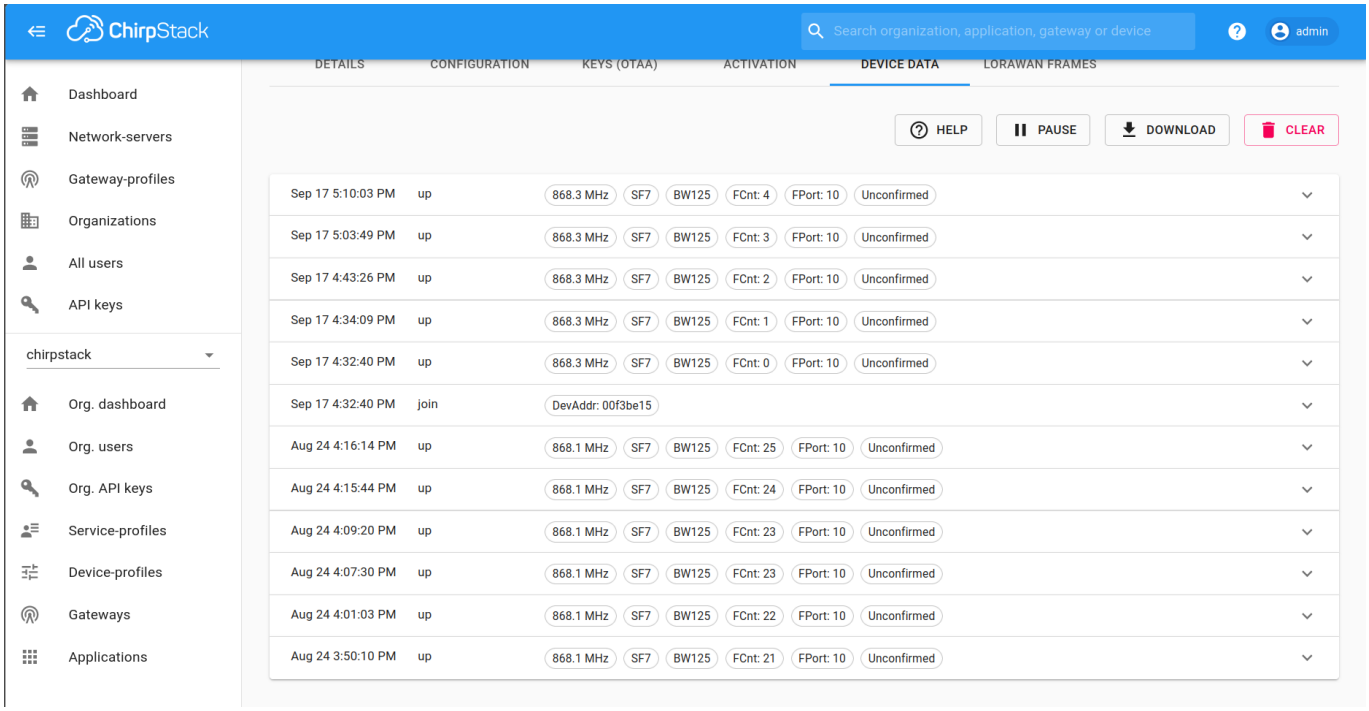


FIGURE 3.3 – Réception des données du device dans l'interface ChirpStack.

Procédure de join OTAA : Avant sa première communication, le dispositif effectue une procédure de *join* au cours de laquelle le **DevEUI** et l'**AppKey** sont vérifiés par le network server. Le DevEUI permet au réseau d'identifier le dispositif, tandis que l'AppKey sert à authentifier la requête de join et à dériver les clés de session. Cette procédure aboutit à la génération d'un **DevAddr**, d'une **AppSKey** et d'une **NwkSKey**, qui seront utilisés par la suite pour la transmission. Une fois le device ajouté, il peut donc rejoindre le réseau et commencer à envoyer des données, qui seront déchiffrées par l'application server à l'aide de l'AppSKey, puis décodées par la fonction définie dans le device-profile.

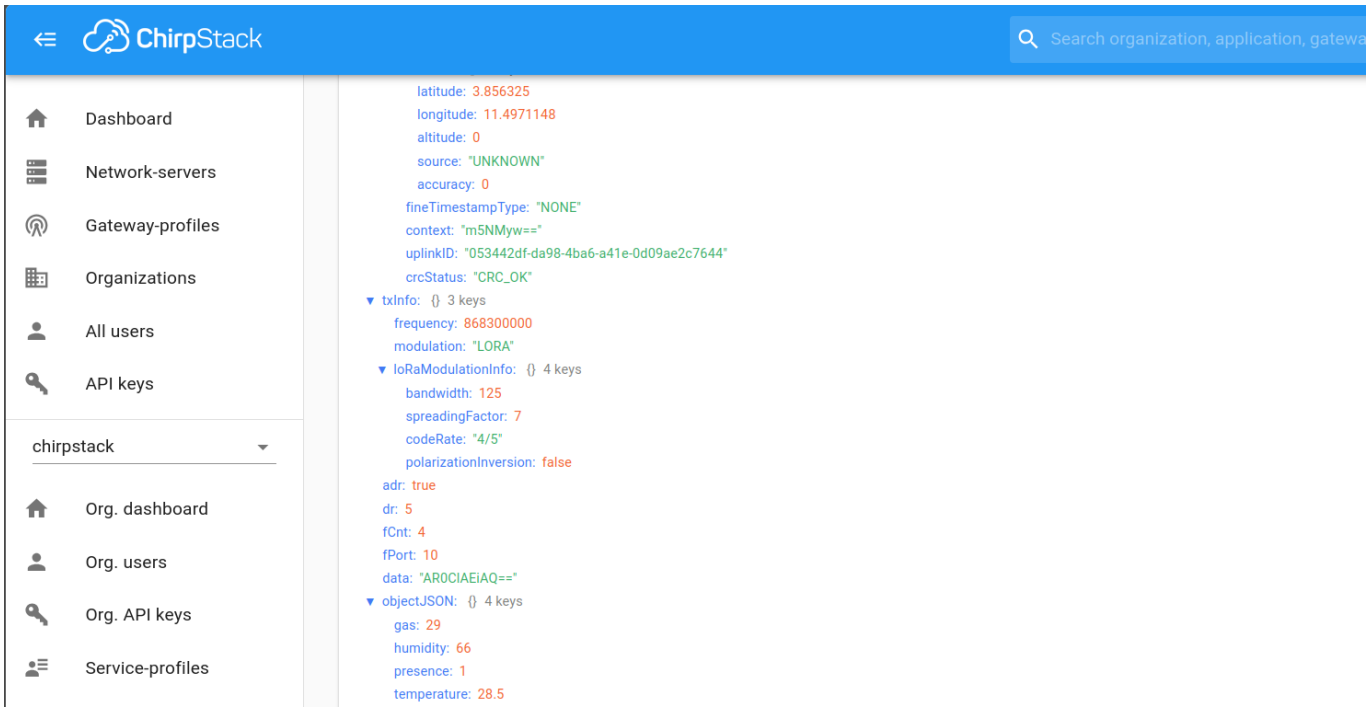


FIGURE 3.4 – Détail d'une transmission LoRaWAN décodée dans ChirpStack.

Activation de l'intégration HTTP

La dernière partie de la configuration de ChirpStack consiste à mettre en place l'intégration HTTP pour l'envoi des données vers Node-RED. Dans l'application créée au préalable, il suffit d'ajouter une intégration HTTP sous l'onglet *Integrations*, puis d'y définir une URL par exemple du type `/chirpstack-network-server/uplink/data` vers laquelle un JSON contenant le payload complet est envoyé via une requête HTTP POST.

Configuration de Node-RED

La configuration de Node-RED commence par un nœud `http in` qui reçoit le payload (HTTP POST) depuis ChirpStack à l'URL définie. Vient ensuite un nœud d'accusé de réception qui envoie une réponse HTTP 200 signalant que le payload a bien été reçu.

Un nœud `function` a ensuite pour rôle de sélectionner les données utiles présentes dans le payload : le DevEUI, la température, l'humidité, le pourcentage de gaz et la présence. Ainsi que d'écrire la requête SQL d'ajout des données dans la base selon le modèle physique de données.

Le nœud `PostgreSQL` réalise l'ajout en base. Il prend comme paramètres les identifiants de connexion au service PostgreSQL, lancé dans le même conteneur Docker : ChirpStack, Node-RED et PostgreSQL sont donc en réseau Docker, ce qui permet à ChirpStack de parler à Node-RED via HTTP et à Node-RED de se connecter au service PostgreSQL par son nom dans le réseau Docker. Comme identifiants de connexion à PostgreSQL, un utilisateur a été défini au lancement de PostgreSQL pour la base de données associée au dispositif.

Un nœud `change` est ensuite nécessaire, car le nœud PostgreSQL ne fonctionne correctement qu'avec une seule requête SQL : le nœud Change remplace la précédente requête SQL par la nouvelle avant l'entrée dans le nœud PostgreSQL final, qui termine ainsi le flux d'insertion des données collectées en base.

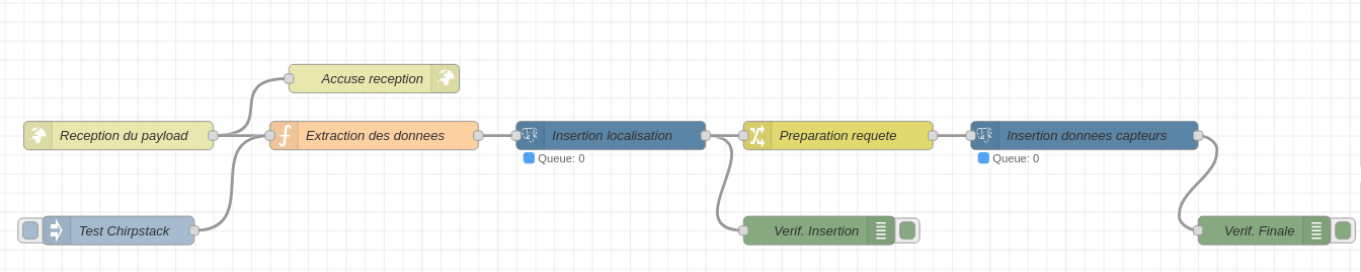


FIGURE 3.5 – Flux d'insertion des données dans Node-RED.

Remarque sur le débogage. Pour le débogage, deux nœuds `debug` sont placés après les nœuds PostgreSQL afin de vérifier si les insertions ont réussi, et un nœud `inject` est placé au tout début pour tester le flux avec un payload JSON similaire à celui de ChirpStack, sans avoir besoin de ce dernier.

À l'issue de cette configuration, les données sont en base et prêtes à être utilisées par l'application.

3.1.8 Visualisation des données en BD

Modèle physique de données

Le stockage des données repose sur trois tables complémentaires. La table `sensor_data` regroupe l'identité de chaque dispositif : son identifiant unique (`dev_eui`), son nom, ainsi que sa position géographique (latitude et longitude). La table `sensor_readings` contient l'ensemble des mesures collectées, chaque enregistrement étant rattaché à un dispositif par une clé étrangère vers `sensor_data` ; elle regroupe l'horodatage de la mesure, les valeurs de température, d'humidité et de gaz, l'état de présence, ainsi que deux indicateurs de qualité radio (RSSI et SNR) renseignés par le réseau LoRaWAN. Enfin, la table `alert_config` centralise les paramètres de seuils (température minimale et maximale, humidité minimale et maximale, concentration maximale de gaz) ainsi que les coordonnées de contact (adresse électronique et numéro de téléphone) utilisées pour l'envoi des alertes.

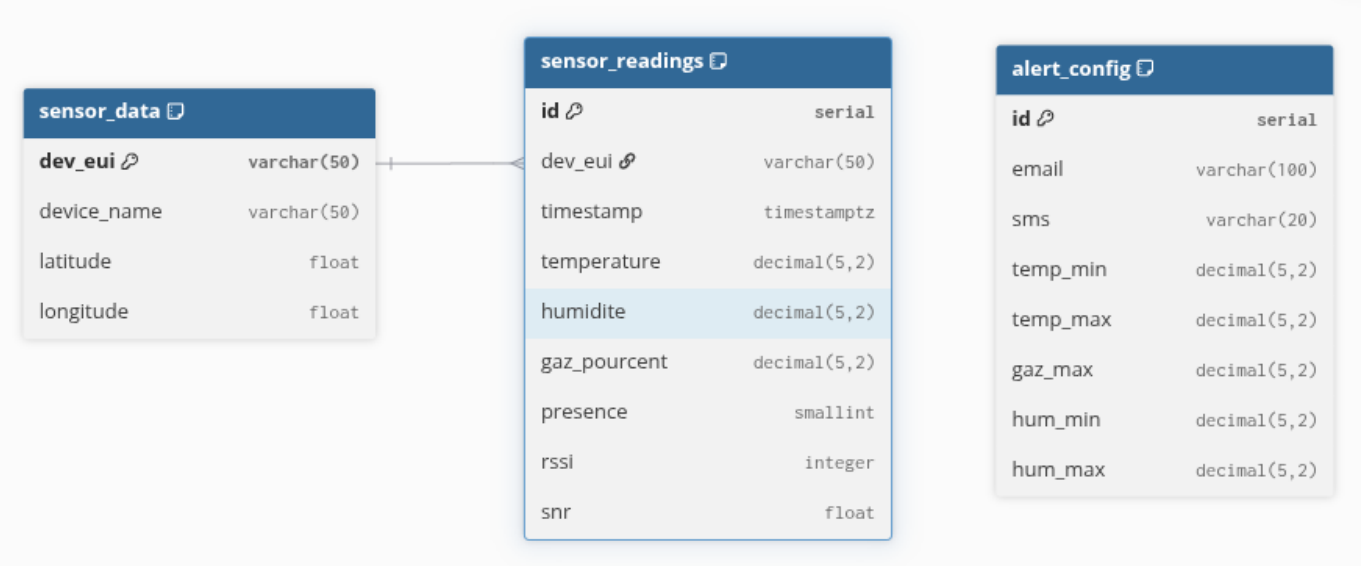


FIGURE 3.6 – Modèle physique de données

3.2 Résultats obtenus

La chaîne complète de collecte, de transmission et de visualisation des données a été validée successivement pour les deux architectures étudiées. Lors de la phase WiFi, les mesures issues des capteurs étaient correctement reçues par le serveur en ligne sous forme de messages structurés au format JSON, avec un envoi périodique toutes les trois minutes. Lors de la phase LoRa/LoRaWAN, les mesures encodées sous forme de trame hexadécimale de quatorze caractères ont été correctement décodées côté ChirpStack, confirmant la cohérence de l'encodage réalisé côté ESP32 (mise à l'échelle par dix, représentation en complément à deux pour la température, ordre des octets en big-endian) avec le décodage attendu côté réseau. La logique de transmission conditionnelle, déclenchant un envoi uniquement en cas de dépassement des seuils définis pour la température, l'humidité ou le gaz, ou en cas de changement de l'état de présence, a permis de réduire significativement le nombre de transmissions radio par rapport à un envoi périodique systématique.

3.2.1 Scénarios de test

Le tableau 3.4 synthétise les principaux scénarios de test réalisés au cours du projet, ainsi que les résultats obtenus par rapport aux résultats attendus.

TABLE 3.4 – Scénarios de test

Scénario	Résultat attendu	Résultat obtenu
Lecture des capteurs et affichage sur le moniteur série	Affichage correct des quatre mesures toutes les secondes	✓ Conforme
Variation de température supérieure au seuil défini	Déclenchement de l'envoi d'une nouvelle trame	✓ Conforme
Absence de changement significatif sur une période prolongée	Aucun envoi de trame durant cette période	✓ Conforme
Encodage / décodage d'une température négative	Reconstruction correcte de la valeur signée côté récepteur	✓ Conforme
Réception d'une trame par la passerelle LoRa	Trame visible dans les journaux d'événements de ChirpStack	✓ Conforme
Transmission de bout en bout vers l'application locale	Affichage de la mesure sur l'interface de visualisation	✓ Conforme

3.2.2 Captures d'écran et résultats visuels

Les figures suivantes illustrent le fonctionnement du système aux différentes étapes de la chaîne de transmission.

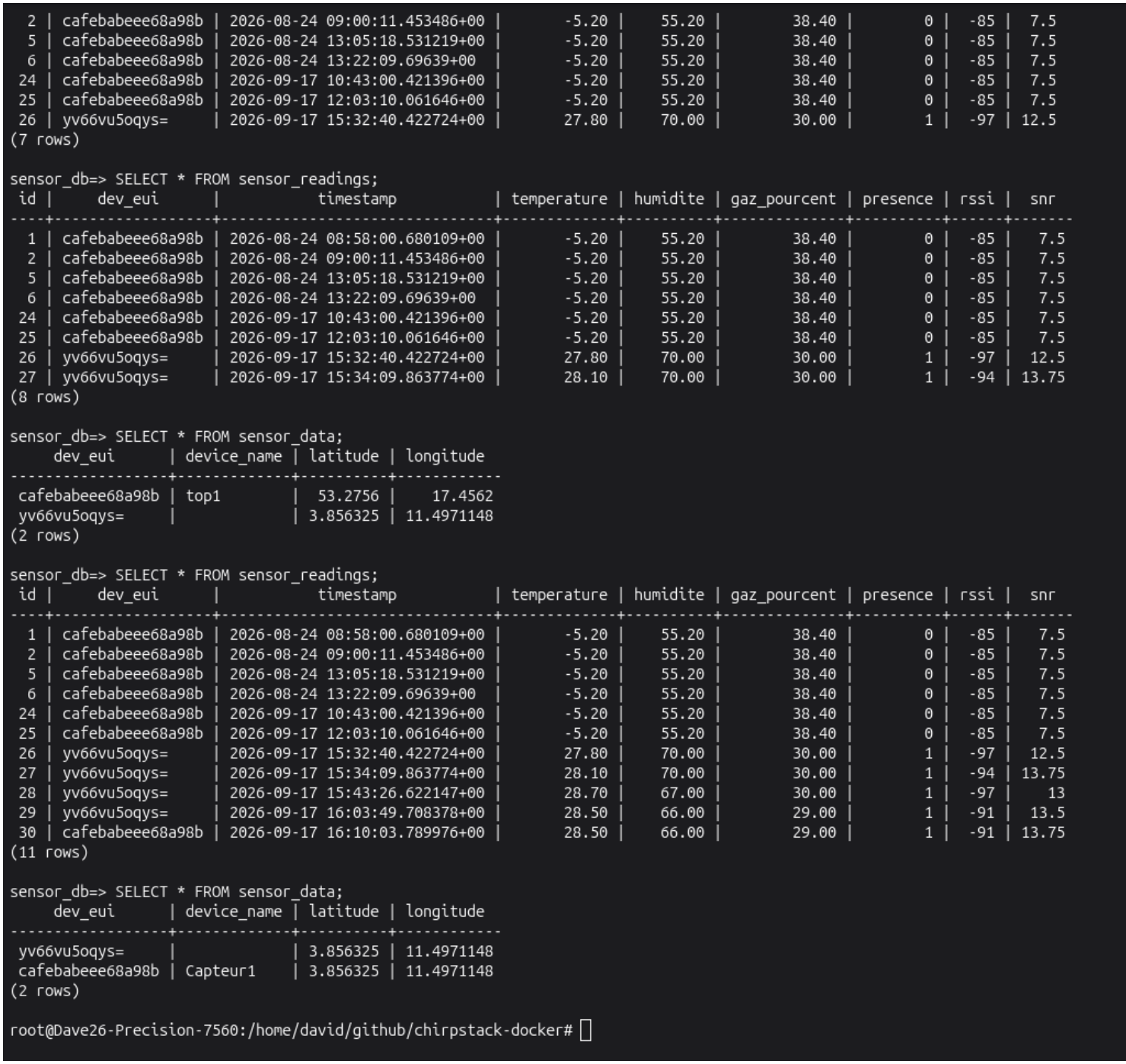


FIGURE 3.7 – Contenu de la base de données PostgreSQL

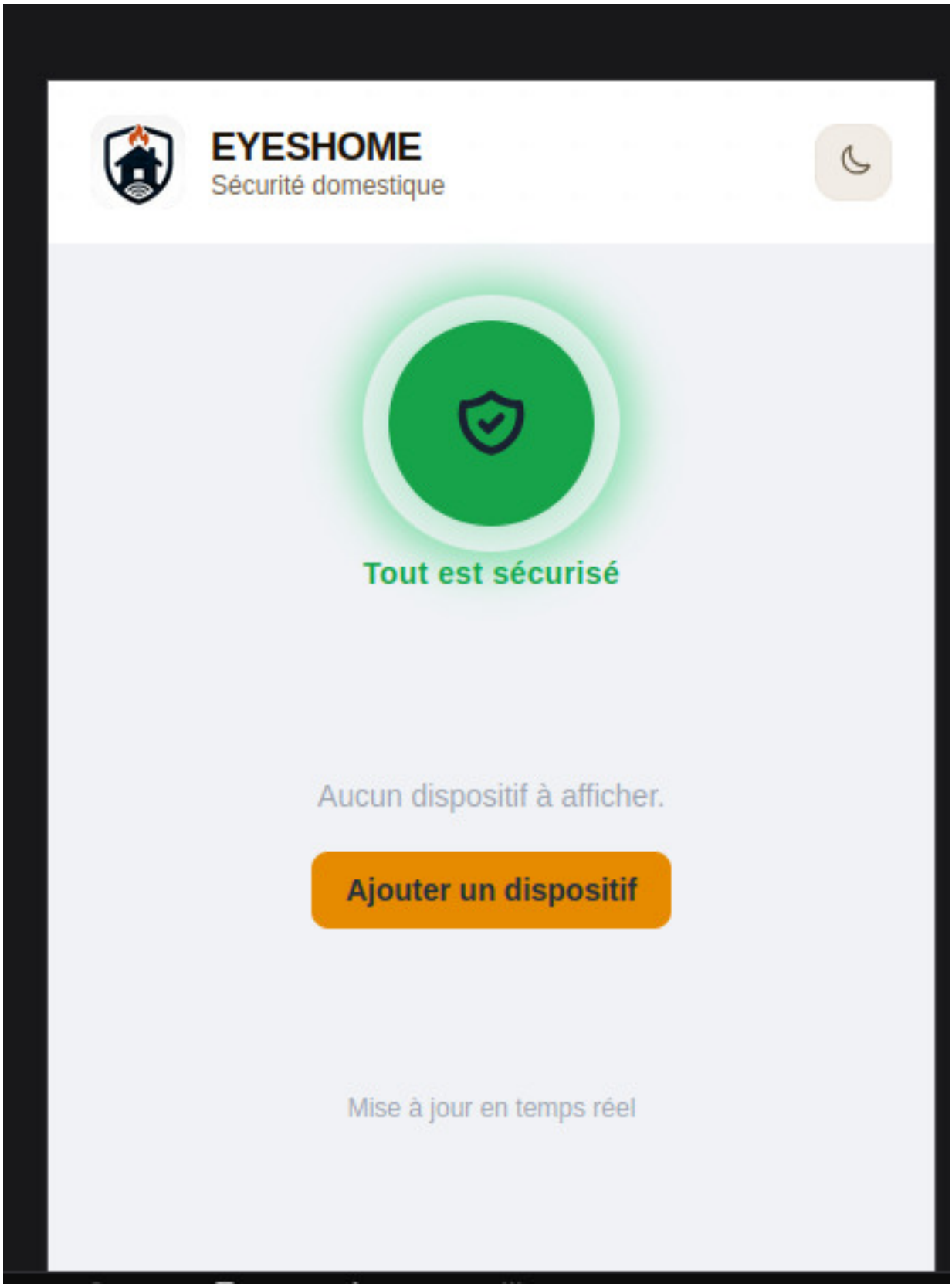


FIGURE 3.8 – Capture d'ecran sans qu'on est aujouter un dispositif

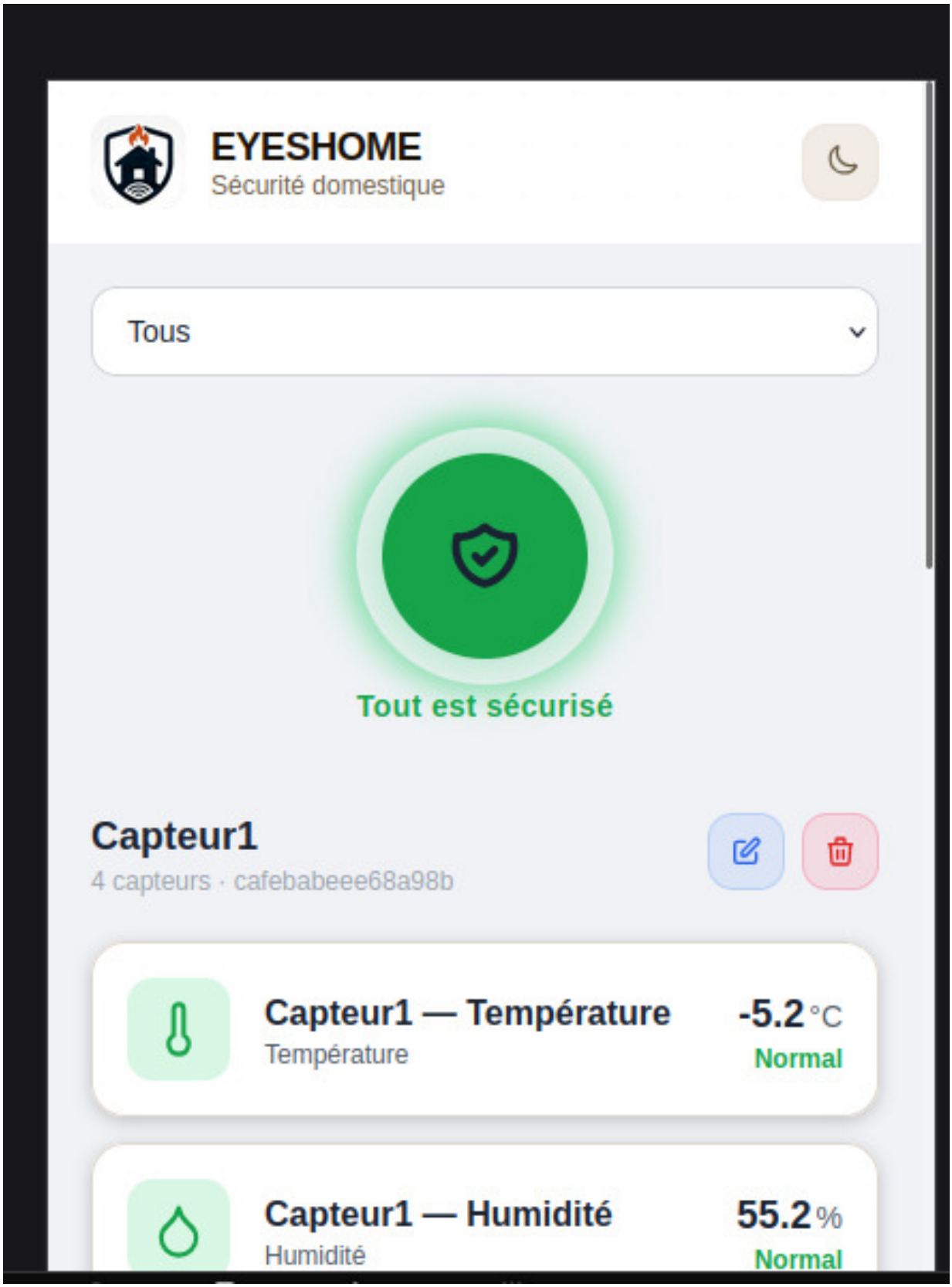


FIGURE 3.9 – Resultat avec le dispositif ajouter-1

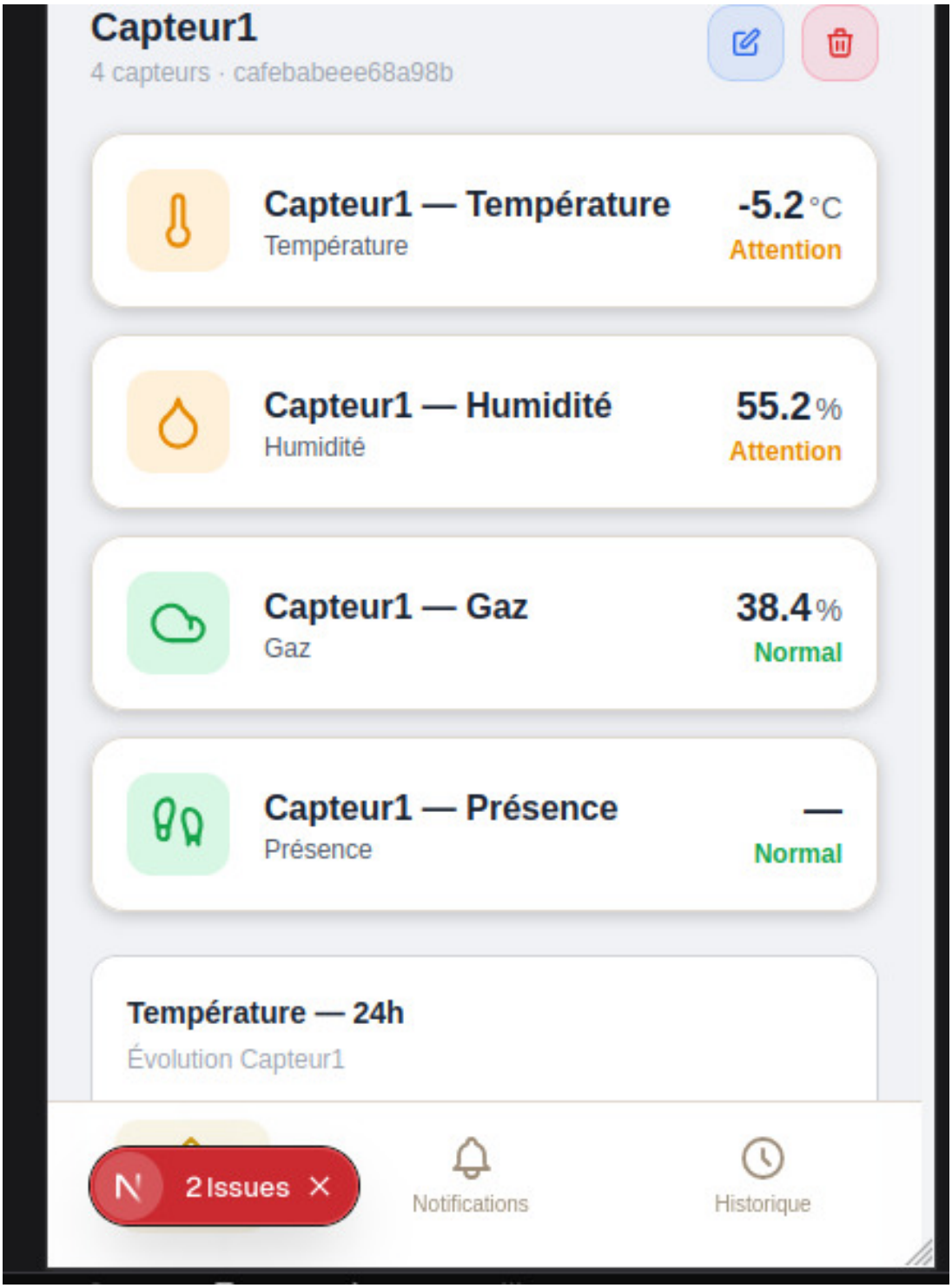


FIGURE 3.10 – Resultat avec le dispositif ajouter-2

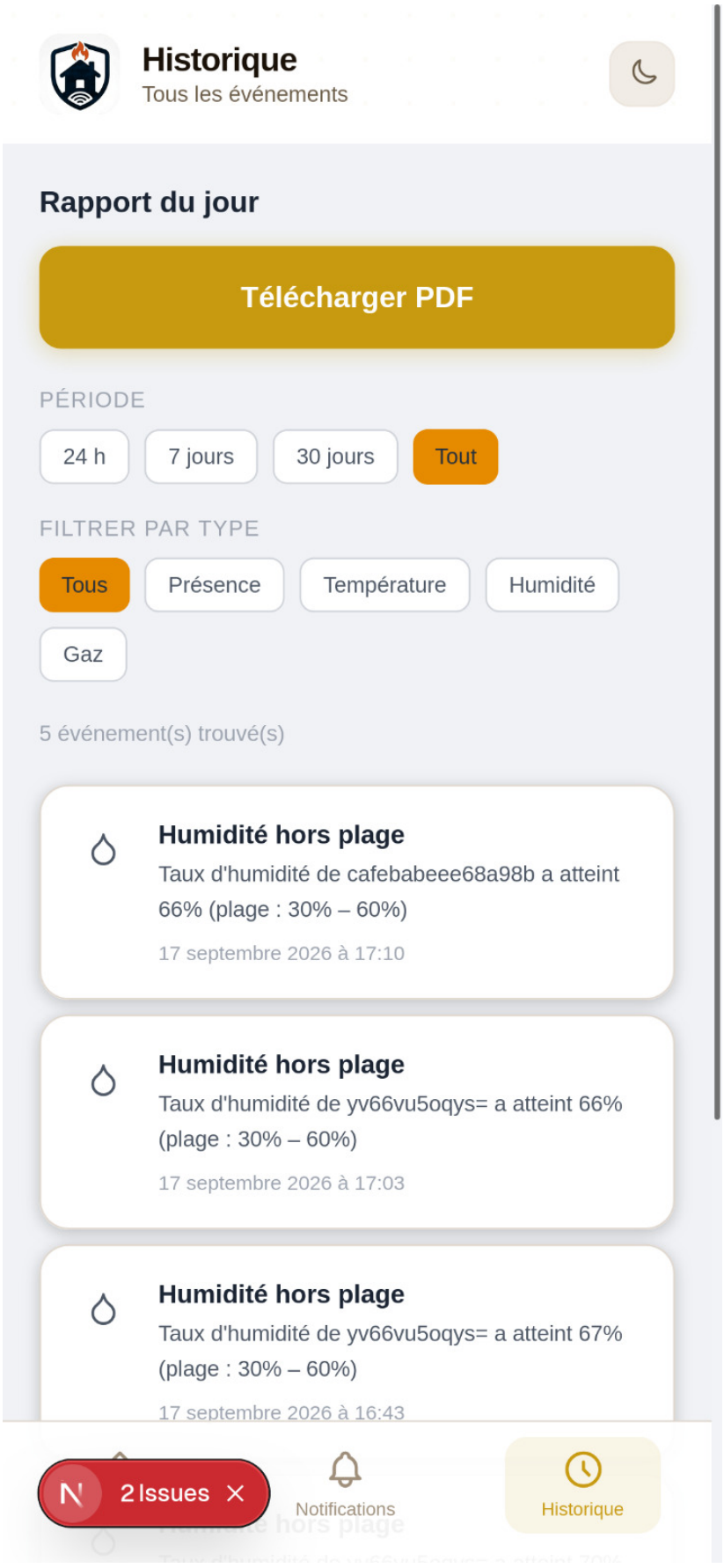


FIGURE 3.11 – Historique

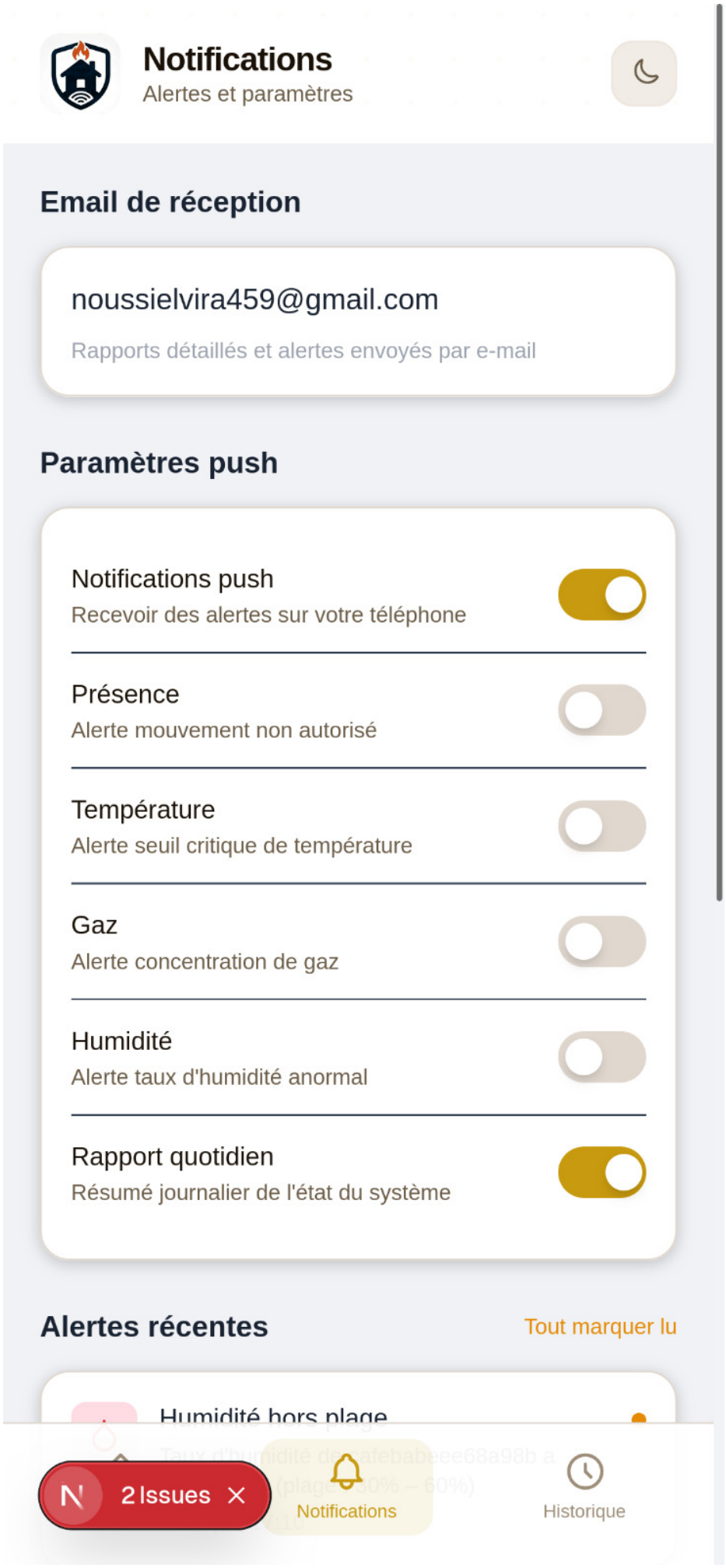


FIGURE 3.12 – Notifications 37

Chapitre 4

Difficultés, limites et perspectives

4.1 Difficultés rencontrées lors de la réalisation du prototype

4.1.1 Partage du port UART entre la console de débogage et la liaison ESP32–Wyres

L'une des principales difficultés rencontrées au cours de ce projet a été la gestion de la communication entre la carte Wyres Base et la carte ESP32. En effet, la carte Wyres Base ne disposant que d'un seul port UART, celui-ci se trouvait partagé entre la console de débogage et la réception des données provenant de l'ESP32, rendant impossible l'utilisation simultanée des deux fonctions. Cette contrainte a nécessité plusieurs approches alternatives, notamment une tentative de basculement vers le protocole I2C. Cette difficulté présente une double nature : elle constitue à la fois une difficulté d'implémentation et une limite intrinsèque de la plateforme matérielle, car l'absence d'un second port UART dédié empêche toute utilisation simultanée de la console de débogage et de la liaison avec l'ESP32.

La solution finalement retenue a consisté à revenir vers l'UART en acceptant la perte de la console de débogage lors des phases de communication, ce qui a permis de stabiliser la liaison sans toutefois résoudre la contrainte matérielle sous-jacente.

4.1.2 Rejet des trames LoRaWAN après redémarrage

La gestion du compteur de trames LoRaWAN (*frame counter*) a posé des problèmes récurrents lors des redémarrages de la carte : le serveur ChirpStack rejetait silencieusement les messages dont le compteur était inférieur à la dernière valeur enregistrée, donnant l'impression que la carte n'émettait plus. Ce phénomène s'explique par le mécanisme de protection contre les attaques par rejeu : le serveur réseau vérifie que chaque trame reçue possède un compteur strictement supérieur au dernier compteur validé. Lorsqu'un dispositif redémarre, son compteur local repart de zéro, ce qui déclenche un avertissement `Frame-counter reset or rollover detected` et conduit au rejet des trames.

La solution choisie a été de désactiver la validation du compteur de trames (*Disable frame-counter validation*) pour le dispositif dans les configurations de ChirpStack. Cette solution, bien que fonctionnelle pour un environnement de test fermé, présente un risque de sécurité puisqu'elle ouvre la porte à des attaques par rejeu.

4.1.3 Insertions multiples en base via Node-RED

Une autre difficulté est apparue lors de la configuration de l'insertion des données en base via Node-RED. Le nœud `node-red-contrib-postgresql` ne supporte pas l'exécution de plusieurs instructions d'insertion dans une seule requête paramétrée, renvoyant l'erreur `cannot insert multiple commands into a prepared statement`.

La solution a consisté en un réusinage du flux : un nœud `change` a été introduit pour préparer chaque requête d'insertion individuellement, et le flux a été dédoublé avec autant de nœuds PostgreSQL que d'insertions nécessaires, au lieu d'un seul nœud tentant de tout exécuter.

4.1.4 Incohérences de structure du payload transmis par l'intégration HTTP

Enfin, une difficulté notable a concerné la structure du payload transmis par l'intégration HTTP de ChirpStack. Le contenu reçu différait légèrement de celui présenté dans l'interface de ChirpStack : l'attribut `objectJSON`, censé contenir les données décodées et formatées, se présentait sous forme de chaîne de caractères brute imitant une structure JSON, et non comme un objet JSON exploitable directement. Par ailleurs, les identifiants tels que le `DevEUI` ou le `GatewayID`, affichés en base hexadécimale dans l'interface, étaient reconvertis en base64 lors de l'envoi HTTP, introduisant une incohérence de représentation.

La solution implémentée a consisté en un ajustement du nœud `function` de Node-RED, chargé de parser la chaîne `objectJSON` et de reconstruire les identifiants dans leur format attendu.

4.2 Limites du dispositif

4.2.1 Restriction du duty cycle LoRaWAN

La limite principale du dispositif réside dans la restriction du *duty cycle* imposée par la carte Wyres Base. Après chaque émission LoRaWAN, la pile Semtech LoRaMac-node impose une période de silence pendant laquelle aucun nouveau message ne peut être transmis. Or, la pile ne met pas automatiquement en file d'attente les messages reçus durant cette période : ils sont simplement perdus. Ceci explique pourquoi seul le premier message envoyé après la mise sous tension atteint effectivement ChirpStack, les suivants étant bloqués par cette restriction.

4.2.2 Précision et portée des capteurs

Sur le plan matériel, plusieurs limites affectent la précision et la portée du système. Le capteur PIR présente une sensibilité spatiale non uniforme : son signal décroît rapidement avec la distance et les angles extrêmes, avec des zones mortes où la détection est inefficace, ce qui peut donner lieu à des faux négatifs ou à des faux positifs. Le capteur MQ-2, quant à lui, est un capteur de gaz à large spectre : il réagit à de nombreux gaz combustibles (CO, alcool, fumée, propane, méthane, butane, hydrogène) sans pouvoir les distinguer les uns des autres. Ainsi, la valeur fournie par le

MQ-2 ne permet d'obtenir qu'une estimation de la concentration globale en gaz détectés dans l'air, sans identification précise du gaz présent, ce qui limite la pertinence des alertes émises. De plus, le capteur MQ-2 présente une dérive de sa ligne de base et des temps de retour lents après exposition à un mélange gazeux.

4.2.3 Broches insuffisantes sur la carte Wyres Base

La carte Wyres Base, enfin, ne dispose pas d'un nombre suffisant de broches avec les modes de communication appropriés pour être utilisée de manière autonome : elle doit être couplée à l'ESP32 pour l'acquisition des capteurs, ce qui introduit la contrainte de partage du port UART évoquée précédemment. Cette limite est inhérente au choix architectural d'utiliser la carte Wyres Base comme module LoRaWAN externe couplé à l'ESP32, plutôt qu'une carte intégrant nativement la réalisation d'envoi de données par LoRaWAN.

4.2.4 Boucle de réactivation de l'alarme

Sur le plan matériel, une limite supplémentaire concerne la proximité physique entre le capteur de présence PIR et le bouton d'arrêt de l'alarme. Lorsqu'un utilisateur tente de désactiver l'alarme en appuyant sur ce bouton, le capteur PIR détecte ce mouvement comme une nouvelle présence et déclenche immédiatement une nouvelle alarme. Ce phénomène crée une boucle de rétroaction où toute tentative de désactivation réactive l'alarme, rendant le bouton d'arrêt inopérant en pratique.

4.2.5 Coût des alertes SMS

Enfin, l'envoi de messages SMS a été envisagé comme moyen d'alerte mais abandonné, car quelle que soit la solution technique envisagée, il implique des coûts récurrents trop élevés pour un prototype à budget limité.

4.3 Perspectives d'évolution

Les perspectives d'amélioration identifiées s'organisent selon un gradient croissant de complexité, allant de la simple substitution de composants à l'évolution vers une plateforme de surveillance multi-dispositifs.

4.3.1 Intégration d'un ESP32 avec module LoRa intégré

Une première perspective consiste à remplacer le couple ESP32–module Wyres LoRa par un microcontrôleur ESP32 intégrant nativement un module LoRa, tel que le LILYGO T3S3 ou le T-Beam Supreme. Cette approche éliminerait la contrainte de partage du port UART entre la console de débogage et la communication avec le module radio, résolvant ainsi la difficulté de transmission et de débogage rencontrée lors du développement. Des cartes comme le T3S3 embarquent un ESP32-S3 couplé à un module SX1262 ou SX1276, offrant une solution compacte pour les applications IoT de surveillance.

4.3.2 Remplacement du bouton poussoir par une télécommande

Pour résoudre le problème de boucle lié à la proximité entre le capteur PIR et le bouton d'arrêt, une perspective consiste à remplacer le bouton poussoir physique par une télécommande permettant la désactivation à distance. L'utilisateur pourrait ainsi arrêter l'alarme sans pénétrer dans la zone de détection du capteur, supprimant le risque de réactivation involontaire. Cette solution s'appuie sur une communication sans fil dédiée (infrarouge, radiofréquence ou Bluetooth) entre la télécommande et le dispositif central.

4.3.3 Utilisation de capteurs adaptés et de meilleure précision

L'amélioration de la fiabilité du système passe par le remplacement des capteurs actuels par des modèles plus adaptés. Pour la détection de gaz, le MQ-2 présente une sensibilité croisée à de nombreux gaz (LPG, butane, méthane, alcool, hydrogène, fumée, CO) sans pouvoir les distinguer, ce qui limite la pertinence des alertes émises [citation :12]. Un capteur dédié à un gaz spécifique offrirait une mesure plus fiable. Pour la détection de présence, le PIR actuel présente une sensibilité spatiale non uniforme, avec une décroissance rapide du signal au-delà de 3 mètres et des zones mortes où la détection échoue. L'utilisation de plusieurs détecteurs ou d'un capteur de meilleure portée permettrait de couvrir efficacement une pièce entière indépendamment de sa taille. Enfin, l'ajout d'une surveillance du niveau de batterie constituerait une amélioration fonctionnelle notable pour anticiper les besoins de maintenance.

4.3.4 Évolution vers un réseau de surveillance multi-dispositifs

À plus long terme, le dispositif pourrait évoluer d'une unité autonome vers un réseau de surveillance couvrant plusieurs pièces ou bâtiments. Chaque nœud déploierait les améliorations précédentes (ESP32 intégré, capteurs adaptés, communication sans fil dédiée) et transmettrait ses mesures vers une plateforme centralisée. Cette architecture permettrait une supervision globale, la corrélation des événements entre différentes zones et une gestion unifiée des alertes. La pile logicielle déjà en place (ChirpStack, Node-RED, PostgreSQL) constitue une base extensible pour accueillir un nombre croissant de dispositifs.

Conclusion

Le projet EyesHome est né du constat que les foyers et petits locaux techniques disposent rarement de solutions de surveillance environnementale à la fois accessibles, autonomes et indépendantes d'une infrastructure cloud tierce. Pour répondre à ce problème, la démarche adoptée a consisté à construire progressivement une chaîne complète de collecte, de transmission et de visualisation de données, en validant d'abord une architecture simple fondée sur le WiFi, avant de la faire évoluer vers une architecture LoRa/LoRaWAN mieux adaptée aux contraintes de portée et de consommation, puis en finalisant l'ensemble du déploiement en environnement local à l'aide de Docker.

Au regard des objectifs fixés en introduction, le prototype matériel intégrant les quatre capteurs autour de l'ESP32 a été réalisé et validé, la chaîne de transmission WiFi initiale a fonctionné comme attendu, la migration vers LoRa/LoRaWAN a été menée à bien avec un encodage des données optimisé pour limiter la taille des trames radio, et le déploiement local de l'ensemble des composants logiciels a été achevé avec succès. Les principales difficultés rencontrées ont concerné la configuration précise des broches de communication série sur le microcontrôleur utilisé, ainsi que la nécessité d'harmoniser strictement les conventions d'encodage (ordre des octets, représentation des valeurs signées) entre le microcontrôleur émetteur et le décodeur du serveur réseau, deux points qui ont nécessité plusieurs itérations de vérification avant d'aboutir à un fonctionnement fiable.

Plusieurs perspectives d'amélioration se dégagent de ce travail. Sur le plan matériel, l'ajout de capteurs complémentaires (qualité de l'air, détection de fumée, niveau sonore) permettrait d'élargir le champ de surveillance du système. Sur le plan logiciel, la mise en place d'un mécanisme d'alerte plus élaboré, notamment par notification push ou message directement adressé à l'utilisateur, renforcerait la réactivité du système face aux situations critiques. Enfin, un déploiement à plus grande échelle, impliquant plusieurs nœuds capteurs communiquant avec une même passerelle, permettrait de valider la capacité du système à superviser simultanément plusieurs zones d'un même bâtiment.

Bibliographie

[1] S. Montagny, *LoRa et LoRaWAN pour l'Internet des Objets*, Université Savoie Mont Blanc, version 8, octobre 2020. [En ligne]. Disponible : https://www.univ-smb.fr/lorawan/

[2] Strataggem, *Micro Gateway LoRaWAN ecoSignal (STGHW-00054) — Manuel de configuration*, Strataggem, 2022. [En ligne]. Disponible : https://wiki.strataggem.com/

[3] CampusIoT, *RIOT-wyres : RIOT OS for Wyres LoRaWAN boards*, GitHub repository, 2022. [En ligne]. Disponible : https://github.com/CampusIoT/RIOT-wyres

[4] ChirpStack, *ChirpStack Open-Source LoRaWAN Network Server Documentation*. [En ligne]. Disponible : https://www.chirpstack.io/docs/

[5] Node-RED Foundation, *Node-RED Documentation*. [En ligne]. Disponible : https://nodered.org/docs/

[6] Espressif Systems, *ESP32 Series Datasheet*, Version 4.5, Espressif Systems, Shanghai, Chine, 2023.

Annexe A

Code source complet

A.1 Code Source Complet ESP32

Listing A.1 – Code complet ESP32 — lecture des capteurs, encodage hexadécimal et envoi UART vers la Wyres LoRa

```
#include <DHT.h>

// ============================================
//  BROCHAGES (   ADAPTER SELON VOTRE CONFIG)
// ============================================
#define DHT_PIN       23    // DHT11 - temp rature/humidit
#define MQ2_PIN       34    // MQ-2 - capteur gaz (ADC1)
#define PIR_PIN       25    // PIR Motion Sensor - pr sence
#define BUTTON_PIN    32    // Bouton poussoir - arr t buzzer
#define BUZZER_PIN    33    // Buzzer actif

//  UART vers Wyres LoRa
//  SP CIFIQUE ESP32 CLASSIQUE (pas S3) :
// GPIO6    GPIO11 sont c bl s en interne    la m moire FLASH du module
//     NE JAMAIS les utiliser comme GPIO, risque de blocage au d marrage.
// On utilise donc GPIO16/17, les pins standards de l'UART2 mat riel.
//  Si ton module utilise une puce ESP32-WROVER (avec PSRAM), GPIO16/17
// peuvent  tre  occup es par la PSRAM     dans ce cas, utiliser GPIO4/GPIO2
//    la place (   tester avec le sketch loopback si besoin).
#define UART_RX_PIN   16    // RX depuis Wyres (reli  au TX de la Wyres)
#define UART_TX_PIN   17    // TX vers Wyres (reli  au RX de la Wyres)
#define UART_BAUD     9600  // D bit UART

// ============================================
//  INITIALISATIONS
// ============================================
DHT dht(DHT_PIN, DHT11);

bool presence = false;
bool buzzerActif = false;

// ----- Seuils de d tection de changement -----
#define SEUIL_TEMP    1.0   //  C   : envoi si  cart  >= 1 C
#define SEUIL_HUM     5.0   // %   : envoi si  cart  >= 5%
#define SEUIL_GAZ     10    // %   : envoi si  cart  >= 10%
// Pr sence : tout changement d' tat  (0<->1) d clenche l'envoi

```

```
38 // ----- Fr quence de lecture des capteurs -----
39 const unsigned long FREQ_LECTURE = 1000; // 1 seconde
40 unsigned long derniereLecture = 0;
41
42 // ----- M morisation des derni res valeurs ENVOY ES -----
43 float temp_precedente = -999;
44 float hum_precedente  = -999;
45 int   gaz_precedent   = -999;
46 bool  presence_precedente = false;
47 bool  premierEnvoi = true;   // force l'envoi de la toute premi re lecture
48
49 // ============================================
50 //  SETUP
51 // ============================================
52 void setup() {
53   Serial.begin(115200);                                    // Debug USB
54   Serial2.begin(UART_BAUD, SERIAL_8N1, UART_RX_PIN, UART_TX_PIN); // UART Wyres
55
56   delay(1000);
57
58   Serial.println("\n========================================");
59   Serial.println("SYSTEME IoT ESP32 -> WYRES LORA (HEX)");
60   Serial.println("========================================");
61
62   pinMode(PIR_PIN, INPUT);
63   pinMode(BUTTON_PIN, INPUT_PULLUP);
64   pinMode(BUZZER_PIN, OUTPUT);
65   pinMode(MQ2_PIN, INPUT);
66
67   digitalWrite(BUZZER_PIN, LOW);
68
69   dht.begin();
70   Serial.println("    DHT11 initialis ");
71   Serial.println("    UART2 Wyres LoRa configur  (9600 baud)");
72   Serial.println("  - TX: GPIO 17  |  RX: GPIO 16");
73   Serial.print("    Seuils de changement :  T e m p "); Serial.print(SEUIL_TEMP, 1)
       ;
74   Serial.print(" C  |  H u m  "); Serial.print(SEUIL_HUM, 1);
75   Serial.print("% |  G a z  "); Serial.print(SEUIL_GAZ); Serial.println("%");
76   Serial.println("========================================");
77   Serial.println("SYSTEME PRET");
78   Serial.println("========================================\n");
79 }
80
81 // ============================================
82 //  LOOP PRINCIPAL
83 // ============================================
84 void loop() {
85   // Respecte la fr quence de lecture (1 seconde)
86   if (millis() - derniereLecture < FREQ_LECTURE) {
```

```
87      return;
88    }
89    derniereLecture = millis();
90
91    // ----- 1. LECTURE DES CAPTEURS -----
92    presence = digitalRead(PIR_PIN) == HIGH;
93
94    float temperature = dht.readTemperature();
95    float humidite = dht.readHumidity();
96
97    int valeurMQ2 = analogRead(MQ2_PIN);
98    int pourcentageGaz = map(valeurMQ2, 0, 4095, 0, 100);
99    pourcentageGaz = constrain(pourcentageGaz, 0, 100);
100
101   // Si le DHT11 renvoie une erreur de lecture, on ignore ce cycle
102   if (isnan(temperature) || isnan(humidite)) {
103     Serial.println(" Erreur lecture DHT11, cycle ignor ");
104     return;
105   }
106
107   // ----- 2. GESTION BUZZER -----
108   if (presence) {
109     digitalWrite(BUZZER_PIN, HIGH);
110     buzzerActif = true;
111   }
112   if (digitalRead(BUTTON_PIN) == LOW && buzzerActif) {
113     digitalWrite(BUZZER_PIN, LOW);
114     buzzerActif = false;
115     Serial.println(" Buzzer arr t  par bouton");
116     delay(300);
117   }
118
119   // ----- 3. AFFICHAGE DEBUG -----
120   Serial.println("----------------------------------------");
121   Serial.print("Temperature  : "); Serial.print(temperature, 1); Serial.println("
          C ");
122   Serial.print("Humidite     : "); Serial.print(humidite, 1); Serial.println(" %"
        );
123   Serial.print("Gaz (MQ-2)   : "); Serial.print(pourcentageGaz); Serial.println("
         %");
124   Serial.print("Presence     : "); Serial.println(presence ? "OUI (1)" : "NON (0)
        ");
125
126   // ----- 4. D TECTION DE CHANGEMENT -----
127   bool changement = premierEnvoi;
128
129   if (fabs(temperature - temp_precedente) >= SEUIL_TEMP) changement = true;
130   if (fabs(humidite - hum_precedente) >= SEUIL_HUM) changement = true;
131   if (abs(pourcentageGaz - gaz_precedent) >= SEUIL_GAZ) changement = true;
132   if (presence != presence_precedente) changement = true;
```

```
133
134   // ----- 5. ENVOI UNIQUEMENT SI CHANGEMENT D TECT  -----
135   if (changement) {
136     envoyerDonneesHexViaUART(temperature, humidite, pourcentageGaz, presence);
137
138     // M moriser les valeurs comme "derni res envoy es"
139     temp_precedente = temperature;
140     hum_precedente = humidite;
141     gaz_precedent = pourcentageGaz;
142     presence_precedente = presence;
143     premierEnvoi = false;
144   } else {
145     Serial.println("    Pas de changement significatif, rien envoy ");
146   }
147 }
148
149 // ============================================
150 //  FONCTION: CONVERTIT UN OCTET (8 bits) EN 2 CARACT RES HEX
151 // ============================================
152 void byteToHex(uint8_t val, char *out) {
153   const char hexChars[] = "0123456789ABCDEF";
154   out[0] = hexChars[(val >> 4) & 0x0F];
155   out[1] = hexChars[val & 0x0F];
156 }
157
158 // ============================================
159 //  FONCTION: CONVERTIT 2 OCTETS (16 bits) EN 4 CARACT RES HEX
160 // ============================================
161 void word16ToHex(uint16_t val, char *out) {
162   byteToHex((val >> 8) & 0xFF, out);      // octet de poids fort
163   byteToHex(val & 0xFF, out + 2);         // octet de poids faible
164 }
165
166 // ============================================
167 //  FONCTION: ENCODAGE + ENVOI HEXAD CIMAL VIA UART
168 // ============================================
169 void envoyerDonneesHexViaUART(float temp, float hum, int gaz, bool motion) {
170   // -----  tape  1 : TEMP RATURE x10 sur 2 OCTETS (compl ment  deux, sign )
        -----
171   // Garde 1 d cimale : 23.5 C  -> 235   |   -5.2 C  -> -52
172   int16_t temp_signed = (int16_t)constrain((int)round(temp * 10), -32768, 32767);
173   uint16_t temp_word = (uint16_t)temp_signed;  // r interpr tation bit  bit =
        compl ment  deux 16 bits
174
175   // -----  tape  2 : HUMIDIT  x10 sur 2 OCTETS (non sign , toujours positif)
        -----
176   uint16_t hum_word = (uint16_t)constrain((int)round(hum * 10.0), 0, 65535);
177
178   // -----  tape  3 : GAZ x10 sur 2 OCTETS (non sign , toujours positif) -----
179   uint16_t gaz_word = (uint16_t)constrain((int)round(gaz * 10.0), 0, 65535);
```

```
180
181   // -----  tape  4 : PRESENCE sur 1 OCTET (0 ou 1) -----
182   uint8_t presence_byte = motion ? 1 : 0;
183
184   // -----  tape  5 : Conversion en caract res hexad cimaux (BIG-ENDIAN) -----
185   char hexTemp[5], hexHum[5], hexGaz[5], hexPresence[3];
186
187   word16ToHex(temp_word, hexTemp);   // octet fort en premier, octet faible en
          second
188   word16ToHex(hum_word, hexHum);
189   word16ToHex(gaz_word, hexGaz);
190   byteToHex(presence_byte, hexPresence);
191
192   hexTemp[4] = '\0';
193   hexHum[4] = '\0';
194   hexGaz[4] = '\0';
195   hexPresence[2] = '\0';
196
197   // -----  tape  6 : Construire la cha ne finale (14 caract res = 7 octets)
          -----
198   // Ordre fixe : TEMP(x10,2o) | HUM(x10,2o) | GAZ(x10,2o) | PRESENCE(1o)
199   String trame = String(hexTemp) + String(hexHum) + String(hexGaz) + String(
          hexPresence);
200
201   // -----  tape  7 : Envoi via UART2 vers Wyres LoRa -----
202   Serial2.println(trame);
203
204   // -----  tape  8 : Debug -----
205   Serial.println("\n========== ENVOI HEX VERS WYRES LORA (BIG-ENDIAN) ==========="
          );
206   Serial.print("Temp x10 : "); Serial.print(temp, 1); Serial.print(" C  -> ");
207   Serial.print(temp_signed); Serial.print(" -> 0x"); Serial.println(hexTemp);
208   Serial.print("Hum  x10 : "); Serial.print(hum, 1); Serial.print("%  -> ");
209   Serial.print(hum_word); Serial.print(" -> 0x"); Serial.println(hexHum);
210   Serial.print("Gaz  x10 : "); Serial.print(gaz); Serial.print("%   -> ");
211   Serial.print(gaz_word); Serial.print(" -> 0x"); Serial.println(hexGaz);
212   Serial.print("Presence : "); Serial.print(motion ? "1" : "0"); Serial.print("
            -> 0x"); Serial.println(hexPresence);
213   Serial.print("Trame compl te envoy e : ");
214   Serial.println(trame);
215   Serial.println("================================================\n");
216
217   // La Wyres LoRa va :
218   // 1. Recevoir cette cha ne hexad cimale de 14 caract res via UART
219   // 2. La reconvertir en 7 octets (TEMP_HI,TEMP_LO, HUM_HI,HUM_LO, GAZ_HI,GAZ_LO
          , PRESENCE)
220   // 3. Envoyer ce payload brut en LoRaWAN (elle conna t  d  j
221   //    son propre DevEUI et sa position lat/long, enregistr s
222   //    dans ChirpStack lors du provisioning du device)
```

```
223   // 4. ChirpStack d code (diviser TEMP/HUM/GAZ par 10, lecture big-endian) ->
          transmet    Node-RED
224 }
```

A.2 Code Source Complet de Wyres Base

Listing A.2 – Code complet de la Wyres Base — réception UART depuis l'ESP32 et transmission LoRaWAN

```
1  /*
2   * main.c     Wyres Base : R ception UART (ESP32)     LoRaWAN
3   *
4   * Fix definitif :
5   *   - File main r duite    1     thread RX bloqu  tant que main n'a pas
        trait
6   *   - msg_send() bloquant uniquement (plus de msg_try_send)
7   *   - Priorit  thread RX plus basse que main
8   *
9   * C blage ESP32     Wyres Base :
10  *   ESP32 TX (GPIO17)     Wyres Base PA10 (RX)
11  *   ESP32 RX (GPIO16)     Wyres Base PA9  (TX)
12  *   GND                   GND commun
13  */
14
15 #include <stdio.h>
16 #include <string.h>
17 #include <stdint.h>
18 #include <stdbool.h>
19
20 #include "thread.h"
21 #include "msg.h"
22 #include "ringbuffer.h"
23 #include "periph/uart.h"
24 #include "ztimer.h"
25 #include "semtech_loramac.h"
26 #include "net/loramac.h"
27
28 /* ========================================================================
29  * SECTION 1     CONFIGURATION
30  * ======================================================================== */
31
32 #define UART_ESP32          UART_DEV(0)
33 #define UART_ESP32_BAUD     9600
34 #define UART_BUFSIZE        1024
35
36 #define MSG_UART_EOL        (1)
37 #define MSG_DATA_READY      (2)
38
```

```
39 #define LORA_MAX_RETRIES    3
40 #define LORA_RETRY_DELAY_MS 5000
41
42 /* ============================================================================
43  * SECTION 2     CL S LORAWAN OTAA
44  * ============================================================================ */
45
46 static uint8_t deveui[LORAMAC_DEVEUI_LEN] = {
47     0xCA, 0xFE, 0xBA, 0xBE, 0xee, 0x68, 0xa9, 0x8b
48 };
49 static uint8_t appeui[LORAMAC_APPEUI_LEN] = {
50     0xCA, 0xFE, 0xBA, 0xBE, 0x00, 0x00, 0x00, 0x00
51 };
52 static uint8_t appkey[LORAMAC_APPKEY_LEN] = {
53     0x84, 0xbd, 0x96, 0x12, 0xec, 0xd8, 0x1c, 0x1e,
54     0xd4, 0x2a, 0xb9, 0x7a, 0x45, 0xb0, 0x4d, 0xef
55 };
56
57 /* ============================================================================
58  * SECTION 3     PARAM TRES LORAWAN
59  * ============================================================================ */
60
61 #define LORAWAN_TX_PORT     10
62 #define LORAWAN_TX_MODE     LORAMAC_TX_UNCNF
63 #define LORAWAN_DATARATE    5
64 #define LORAWAN_ADR         1
65
66 /* ============================================================================
67  * SECTION 4     VARIABLES GLOBALES
68  * ============================================================================ */
69
70 extern semtech_loramac_t loramac;
71
72 static uint32_t tx_count = 0;
73 static uint32_t tx_ok    = 0;
74 static uint32_t tx_fail  = 0;
75
76 static ringbuffer_t uart_rb;
77 static char         uart_rb_mem[UART_BUFSIZE];
78
79 static char     rx_buf[256];
80 static uint16_t rx_buf_len = 0;
81
82 static char     tx_buf[256];
83 static uint16_t tx_buf_len = 0;
84
85 static uint8_t  payload_buf[7];
86 static uint8_t  payload_len = 0;
87
88 static kernel_pid_t main_pid;
```

```
89  static kernel_pid_t rx_pid;
90
91  static char rx_stack[THREAD_STACKSIZE_DEFAULT + 256];
92
93  /* ============================================================================
94   * SECTION 5     ISR UART
95   * ============================================================================ */
96
97  static void uart_rx_cb(void *arg, uint8_t byte)
98  {
99      (void)arg;
100     ringbuffer_add_one(&uart_rb, (char)byte);
101     if (byte == '\n') {
102         msg_t m;
103         m.type          = MSG_UART_EOL;
104         m.content.value = 0;
105         msg_send_int(&m, rx_pid);
106     }
107 }
108
109 /* ============================================================================
110  * SECTION 6     THREAD RX
111  *
112  * Priorit  THREAD_PRIORITY_MAIN + 1 (plus basse que main)
113  * msg_send() bloquant : le thread RX attend que main soit pr t
114  *     aucun message perdu, aucun d bordement de file
115  * ============================================================================ */
116
117 static void *rx_thread_func(void *arg)
118 {
119     (void)arg;
120
121     msg_t msg_queue[32];
122     msg_init_queue(msg_queue, 32);
123
124     msg_t msg;
125
126     while (1) {
127         msg_receive(&msg);
128         if (msg.type != MSG_UART_EOL) continue;
129
130         /* Assembler le message */
131         rx_buf_len = 0;
132         int c;
133         while ((c = ringbuffer_get_one(&uart_rb)) != -1) {
134             if (rx_buf_len >= sizeof(rx_buf) - 1) {
135                 while (ringbuffer_get_one(&uart_rb) != -1) {}
136                 rx_buf_len = 0;
137                 break;
138             }
```

```
139             rx_buf[rx_buf_len++] = (char)c;
140             if ((char)c == '\n') break;
141         }
142         rx_buf[rx_buf_len] = '\0';
143 
144         /* Supprimer '\r' '\n' */
145         while (rx_buf_len > 0 &&
146                (rx_buf[rx_buf_len-1] == '\n' ||
147                 rx_buf[rx_buf_len-1] == '\r')) {
148             rx_buf[--rx_buf_len] = '\0';
149         }
150 
151         /* V rifier longueur */
152         if (rx_buf_len < 14) {
153             printf("[RX] Trop court (%u), ignore\n", rx_buf_len);
154             continue;
155         }
156 
157         /* V rifier hex */
158         bool valid = true;
159         for (uint16_t i = 0; i < rx_buf_len; i++) {
160             char ch = rx_buf[i];
161             if (!((ch >= '0' && ch <= '9') ||
162                   (ch >= 'A' && ch <= 'F') ||
163                   (ch >= 'a' && ch <= 'f'))) {
164                 valid = false;
165                 break;
166             }
167         }
168         if (!valid) {
169             printf("[RX] Non-hex, ignore : %s\n", rx_buf);
170             continue;
171         }
172 
173         printf("[RX] Valide : %s\n", rx_buf);
174 
175         /*
176          * FIX DEFINITIF : msg_send() BLOQUANT uniquement
177          * Le thread RX attend ici que le thread main soit dans msg_receive()
178          *      garantit qu'aucun message n'est perdu
179          *      la file main de taille 1 suffit
180          */
181         msg_t notify;
182         notify.type          = MSG_DATA_READY;
183         notify.content.value = rx_buf_len;
184         msg_send(&notify, main_pid);  /* bloquant */
185     }
186 
187     return NULL;
188 }
```

```
189
190 /* ============================================================================
191  * SECTION 7     CONVERSION HEX     BYTES
192  * ============================================================================ */
193
194 static uint8_t hex2byte(const char *hex)
195 {
196     uint8_t result = 0;
197     for (int i = 0; i < 2; i++) {
198         result <<= 4;
199         char c = hex[i];
200         if      (c >= '0' && c <= '9') result |= c - '0';
201         else if (c >= 'A' && c <= 'F') result |= c - 'A' + 10;
202         else if (c >= 'a' && c <= 'f') result |= c - 'a' + 10;
203     }
204     return result;
205 }
206
207 static bool build_payload(void)
208 {
209     memcpy(tx_buf, rx_buf, rx_buf_len + 1);
210     tx_buf_len = rx_buf_len;
211
212     payload_len = 0;
213     for (uint16_t i = 0; i + 1 < tx_buf_len; i += 2) {
214         payload_buf[payload_len++] = hex2byte(&tx_buf[i]);
215         if (payload_len >= sizeof(payload_buf)) break;
216     }
217
218     if (payload_len == 0) {
219         printf("[PAYLOAD] Erreur conversion\n");
220         return false;
221     }
222
223     int16_t  temp = (int16_t)((payload_buf[0] << 8) | payload_buf[1]);
224     uint16_t hum  = (uint16_t)((payload_buf[2] << 8) | payload_buf[3]);
225     uint16_t gaz  = (uint16_t)((payload_buf[4] << 8) | payload_buf[5]);
226     uint8_t  pres =  payload_buf[6];
227
228     printf("[PAYLOAD] %u octets :", payload_len);
229     for (int i = 0; i < payload_len; i++) printf(" %02X", payload_buf[i]);
230     printf("\n");
231     printf("[PAYLOAD] Temp=%d.%dC  Hum=%u.%u%%  Gaz=%u%%  Pres=%s\n",
232            temp / 10, temp >= 0 ? temp % 10 : -(temp % 10),
233            hum / 10, hum % 10,
234            gaz / 10,
235            pres ? "OUI" : "NON");
236
237     return true;
238 }
```

```
239
240 /* ============================================================================
241  * SECTION 8     LORAWAN
242  * ============================================================================ */
243
244 static void lorawan_setup(void)
245 {
246     printf("[LORA] Configuration...\n");
247     semtech_loramac_set_deveui(&loramac, deveui);
248     semtech_loramac_set_appeui(&loramac, appeui);
249     semtech_loramac_set_appkey(&loramac, appkey);
250     semtech_loramac_set_dr(&loramac,      LORAWAN_DATARATE);
251     semtech_loramac_set_adr(&loramac,     LORAWAN_ADR);
252     semtech_loramac_set_tx_mode(&loramac, LORAWAN_TX_MODE);
253     semtech_loramac_set_tx_port(&loramac, LORAWAN_TX_PORT);
254     printf("[LORA] DevEUI : %02X:%02X:%02X:%02X:%02X:%02X:%02X:%02X\n",
255            deveui[0], deveui[1], deveui[2], deveui[3],
256            deveui[4], deveui[5], deveui[6], deveui[7]);
257 }
258
259 static void lorawan_join(void)
260 {
261     if (semtech_loramac_is_mac_joined(&loramac)) {
262         printf("[LORA] Deja joint\n");
263         return;
264     }
265     while (!semtech_loramac_is_mac_joined(&loramac)) {
266         printf("[LORA] Join OTAA...\n");
267         uint8_t res = semtech_loramac_join(&loramac, LORAMAC_JOIN_OTAA);
268         if (res == SEMTECH_LORAMAC_JOIN_SUCCEEDED) {
269             printf("[LORA] Join reussi !\n");
270             semtech_loramac_save_config(&loramac);
271             return;
272         }
273         printf("[LORA] Echec join (code=%d), retry 30s\n", res);
274         ztimer_sleep(ZTIMER_SEC, 30);
275     }
276 }
277
278 static bool lorawan_send(void)
279 {
280     if (!semtech_loramac_is_mac_joined(&loramac)) {
281         printf("[LORA] Non joint     rejoin\n");
282         lorawan_join();
283     }
284
285     for (int attempt = 1; attempt <= LORA_MAX_RETRIES; attempt++) {
286         printf("[LORA] Envoi #%lu (tentative %d/%d)...\n",
287                (unsigned long)tx_count, attempt, LORA_MAX_RETRIES);
288
```

```
289         uint8_t res = semtech_loramac_send(&loramac, payload_buf, payload_len);
290
291         switch (res) {
292             case SEMTECH_LORAMAC_TX_DONE:
293                 printf("[LORA] TX OK !\n");
294                 tx_ok++;
295                 return true;
296
297             case SEMTECH_LORAMAC_DUTYCYCLE_RESTRICTED:
298                 printf("[LORA] Duty cycle     attente 30s\n");
299                 ztimer_sleep(ZTIMER_SEC, 30);
300                 break;
301
302             case SEMTECH_LORAMAC_BUSY:
303                 printf("[LORA] MAC busy     attente 5s\n");
304                 ztimer_sleep(ZTIMER_MSEC, LORA_RETRY_DELAY_MS);
305                 break;
306
307             case SEMTECH_LORAMAC_NOT_JOINED:
308                 printf("[LORA] Non joint     rejoin\n");
309                 lorawan_join();
310                 break;
311
312             case SEMTECH_LORAMAC_TX_ERROR:
313                 printf("[LORA] Erreur TX     attente 5s\n");
314                 ztimer_sleep(ZTIMER_MSEC, LORA_RETRY_DELAY_MS);
315                 break;
316
317             default:
318                 printf("[LORA] Erreur inconnue (code=%d)\n", res);
319                 ztimer_sleep(ZTIMER_MSEC, LORA_RETRY_DELAY_MS);
320                 break;
321         }
322     }
323
324     printf("[LORA] Echec apres %d tentatives\n", LORA_MAX_RETRIES);
325     tx_fail++;
326     return false;
327 }
328
329 /* ============================================================================
330  * SECTION 9     MAIN
331  * ============================================================================ */
332
333 int main(void)
334 {
335     printf("\n=== Wyres Base     UART ESP32     LoRaWAN ===\n\n");
336
337     main_pid = thread_getpid();
338
```

```
339     /*
340      * FIX DEFINITIF : file de taille 1
341      * Le thread RX est bloqu  dans msg_send() tant que
342      * le thread main n'est pas dans msg_receive()
343      *     synchronisation parfaite, aucun message perdu
344      */
345     msg_t main_queue[1];
346     msg_init_queue(main_queue, 1);
347 
348     ringbuffer_init(&uart_rb, uart_rb_mem, UART_BUFSIZE);
349 
350     /*
351      * FIX DEFINITIF : priorit  THREAD_PRIORITY_MAIN + 1
352      * Plus basse que main     main s'ex cute en premier
353      * et revient dans msg_receive() avant que RX puisse envoyer
354      */
355     rx_pid = thread_create(rx_stack,
356                            sizeof(rx_stack),
357                            THREAD_PRIORITY_MAIN + 1,
358                            0,
359                            rx_thread_func,
360                            NULL,
361                            "rx");
362     printf("[MAIN] Thread RX pid=%d\n", rx_pid);
363 
364     if (uart_init(UART_ESP32, UART_ESP32_BAUD, uart_rx_cb, NULL) != UART_OK) {
365         printf("[UART] Erreur init !\n");
366     } else {
367         printf("[UART] OK (%d baud)\n", UART_ESP32_BAUD);
368     }
369 
370     lorawan_setup();
371     lorawan_join();
372 
373     printf("\n[OK] Pret. En attente ESP32...\n\n");
374 
375     msg_t msg;
376     while (1) {
377         msg_receive(&msg);
378 
379         if (msg.type != MSG_DATA_READY) continue;
380 
381         if (!build_payload()) continue;
382 
383         tx_count++;
384         lorawan_send();
385 
386         printf("[STAT] TX=%lu  OK=%lu  FAIL=%lu\n\n",
387                (unsigned long)tx_count,
388                (unsigned long)tx_ok,
```

```
389                 (unsigned long)tx_fail);
390     }
391
392     return 0;
393 }
```

Le code source complet de l'ESP32 (fichier `.ino`) et du module Wyres (fichier `.c`) est joint séparément au présent rapport.